ECE595 Reinforcement Learning Course Project Report : Reinforcement Learning from Human Feedback Algorithm for Robotics

Weizheng Wang, Nianyao Du, Kareem Hassan, Silas Hokanson
Purdue University

***Abstract*— Reinforcement learning (RL) has demonstrated strong capability in training robots for locomotion and navigation, yet its practical deployment often depends on carefully shaped reward functions. In many real-world robotics objectives, such as smoothness, comfort, naturalness, and social compliance handcrafted rewards can be incomplete or misaligned, causing unstable training and occasional reward exploitation. Reinforcement Learning from Human Feedback (RLHF) offers an alternative by learning a reward model from preference comparisons over short trajectory segments, aiming to optimize behaviors that better match human judgments. In this work, we investigate RLHF as a reward-engineering substitute by combining preference-based reward learning (e.g., Bradley-Terry Model) with an off-policy continuous control learner based on Soft Actor-Critic (SAC). We benchmark standard SAC against RLHF-augmented training on diverse robotics tasks including bipedal and quadrupedal locomotion (Humanoid, Walker2D, Ant) and socially aware 2D navigation. Our results show that RLHF can effectively improve reward learning performance yielding a smoother, more informative learning signal and faster progress under the learned reward once supervision becomes consistent. However, overall task performance is strongly bounded by feedback quality: when preferences are provided by a scripted/programmatic supervisor rather than true human evaluators, the learned reward may only partially reflect the task objective, especially for complex behaviors such as humanoid walking. In simpler settings (e.g., Walker stand-up), where scripted preferences align more closely with task success, RLHF achieves performance comparable to SAC.**

***Index Terms*— Reinforcement Learning from Human Feedback, Reinforcement Learning, Robotics.**

Thanks to Prof. Ghasemi for the excellent teaching in the Fall 2025 ECE 595-RL course, this report is based on the work completed throughout the semester.

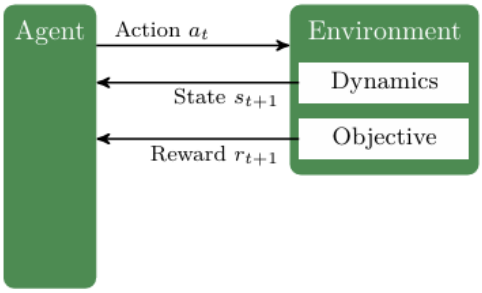
I. Introduction

Reinforcement learning (RL) has become a prominent paradigm for training robots to perform complex behaviors, including locomotion, manipulation, and navigation. By optimizing a policy through interaction, RL can discover control strategies that are difficult to design manually and can adapt to high-dimensional dynamics. However, despite impressive results in simulation and increasing success in real systems, RL in robotics remains heavily constrained by reward design. Many practical objectives such as energy efficiency, smooth and human-comfortable motion, natural-looking behaviors, safety margins, and social compliance are hard to fully specify with a single scalar reward. As a result, reward functions are often hand-tuned combinations of heuristics and penalties, which can be time-consuming to engineer, sensitive to hyperparameters, and prone to unintended solutions. This mismatch between what is rewarded and what is desired can lead to unstable learning or "reward hacking," where the agent exploits loopholes in the reward rather than accomplishing the intended task.

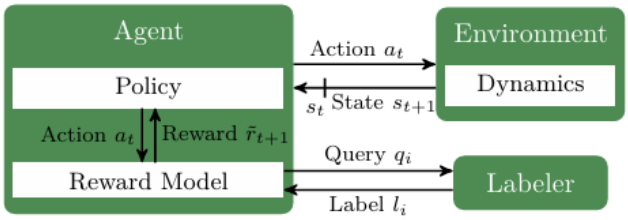
Reinforcement Learning from Human Feedback (RLHF) offers a complementary approach that reduces dependence on hand-crafted rewards by learning the reward signal from preferences. Instead of specifying a dense reward, a supervisor provides pairwise comparisons between short trajectory segments, and a reward model is trained to explain these preferences. The policy then optimizes this learned reward via standard RL. In domains such as language modeling, RLHF has shown that preference supervision can better align optimized behaviors with human intent. For robotics, RLHF is especially attractive because many desiderata are naturally expressed as qualitative judgments, even when they are difficult to encode as analytic reward terms.

Despite this promise, deploying RLHF for robotics introduces new challenges. Human feedback is costly, noisy, and may change over time, while reward models can drift as the policy distribution shifts. Moreover, in many early-stage robotics experiments, "human feedback" is approximated by scripted or programmatic supervisors that emulate preferences automatically. This enables scalable experimentation, but it also creates a critical limitation: the learned reward can only be as good as the proxy supervisor. When the proxy is biased, sparse, or weakly correlated with the true task objective, RLHF may learn a smooth and consistent reward that is nonetheless misaligned with real performance an effect that is most visible in high-dimensional, contact-rich tasks such as humanoid locomotion.

In this work, we study RLHF as a practical alternative to reward engineering for continuous-control robotics by pairing preference-based reward learning (Bradley Terry style modeling) with an off-policy RL backbone based on Soft Actor-Critic (SAC). We evaluate standard SAC and RLHF-augmented training on a range of benchmarks spanning bipedal and quadrupedal locomotion (Humanoid, Walker2D, Ant), as well as socially aware 2D navigation. Our experiments highlight a consistent pattern: RLHF can improve reward learning effectiveness, producing a smoother and more informative learning signal and accelerating progress under the learned reward once supervision becomes consistent. However, overall task return depends strongly on feedback quality. In our current setup, where supervision is generated by a scripted program, RLHF exhibits clear



(a) The standard RL setting.



(b) RLHF with reward modeling

Fig. 1: Contrasting the standard RL setting with RLHF in its most common formulation, using a reward model. In each step, the policy commits to an action at and receives the next state st+1 and either the true reward r_{t+1} or an estimate r_{t+1} in return (symbolized by $\hat{r}_{t+1}$). In contrast to the standard RL setting, the true reward function is not known in the RLHF setting but instead learned form human feedback. This reward learning process is decoupled from policy learning and can happen fully asynchronously. The dataset consists of a set of queries qi (e.g., pairs of trajectory fragments) and their labels li (e.g., a preference for one of the fragments).

performance gaps on difficult tasks such as Humanoid-Walk, while achieving performance close to SAC on simpler tasks such as Walker-Standup where proxy preferences align more directly with success.

The contributions of this report can be summarized as follows:

1. We implement an RLHF pipeline for continuous-control robotics using preference-based reward modeling integrated with SAC;

2. We provide a comparative evaluation of RLHF and standard RL across diverse locomotion and navigation benchmarks, analyzing training stability and sample efficiency;

3. We empirically characterize the impact of feedback quality by contrasting RLHF outcomes on challenging (Humanoid-Walk) versus simpler (Walker-Standup) tasks under scripted supervision, and discuss implications for moving toward real human feedback.

II. Motivation

Robotic systems deployed in real-world environments increasingly rely on reinforcement learning (RL) to acquire complex motor skills and navigation behaviors. While modern RL algorithms such as Soft Actor-Critic (SAC) have demonstrated strong performance in continuous control tasks, their success is heavily dependent on carefully designed reward functions. In many robotics applications, especially those involving interaction with humans or unstructured environments, specifying a reward that fully captures desirable behavior is difficult, time-consuming, and often incomplete.

In practice, small changes or inaccuracies in reward design can lead to unintended behaviors, unstable training, or policies that optimize the reward while violating intuitive or human-preferred behavior. This issue becomes more pronounced in tasks where qualitative aspects such as smoothness, comfort, naturalness, or social acceptability matter, as these properties are difficult to encode mathematically. As a result, purely reward-driven RL may converge to solutions that are optimal numerically but unsatisfactory from a human perspective.

Reinforcement Learning from Human Feedback (RLHF) provides a promising alternative by incorporating human preferences directly into the learning process. Instead of relying solely on a handcrafted reward signal, RLHF learns a reward model from human comparisons of agent behavior, allowing the training objective to better align with human judgment. This paradigm has shown strong results in domains where explicit reward specification is challenging, motivating its application to robotics control and navigation tasks.

The motivation of this project is to systematically study and compare the training performance, stability, and behavioral outcomes of standard RL methods and RLHF-based methods across a range of robotics environments. By evaluating these approaches on locomotion and navigation tasks, this work aims to better understand when human feedback provides meaningful advantages over traditional reward engineering, and how RLHF can improve robustness and alignment in robotic learning systems.

III. Background

Reinforcement learning has become a central framework for robotic control due to its ability to learn policies directly from interaction with the environment. In continuous control settings, actor critic methods have been particularly successful, as they combine value-based learning with direct policy optimization. Among these methods, Soft Actor-Critic (SAC) has emerged as a state-of-the-art algorithm due to its entropy-regularized objective, which encourages exploration and leads to improved training stability and sample efficiency.

SAC optimizes a maximum-entropy objective, balancing reward maximization with policy stochasticity. This formulation is especially well-suited for robotic tasks that involve high-dimensional state and action spaces, such as humanoid

locomotion or quadrupedal walking. As a result, SAC has been widely adopted in benchmark robotics environments and serves as a strong baseline for evaluating learning performance.

Despite these advances, traditional RL methods fundamentally depend on the assumption that the environment provides a well-defined and informative reward function. In many robotics scenarios, however, the reward is only a proxy for the desired behavior and may fail to capture important qualitative aspects. This limitation has motivated research into alternative learning paradigms that reduce reliance on manually designed rewards.

Reinforcement Learning from Human Feedback addresses this challenge by replacing or augmenting the environment reward with a learned reward model derived from human preferences. Typically, humans are asked to compare short trajectory segments and indicate which behavior they prefer. These comparisons are used to train a reward model, often using probabilistic preference models such as the Bradley Terry formulation. The learned reward is then used to optimize the policy using standard RL algorithms.

Recent advances have made RLHF more practical for robotics, including methods that reduce the amount of required human feedback and stabilize optimization. By combining the strengths of entropy-regularized RL algorithms like SAC with preference-based reward learning, RLHF offers a framework that maintains strong control performance while improving alignment with human expectations. This project builds on these ideas to evaluate the effectiveness of RLHF across multiple robotic control and navigation tasks.

IV. Literature Review

Ziegler et al. applied reward learning natural language tasks in the paper In Fine-Tuning Language Models from Human Preferences [1]. This paper introduced the idea of letting humans compare outputs and essentially say which one they prefer. Some issues encountered were the problem of copying, the modelers penalized inaccuracies but not copying, and so the models learned to copy their sources directly.

Stiennon et al. improved summary quality by training a model to optimize for human preferences in the paper Learning to summarize from human feedback [2]. In this paper they drastically improved summary quality over the previous paper, as well as reduced the copying problem. However, part of the reason they were able to do so was a drastic increase in model size. As a result, a major limitation of this was the extreme computational cost. Another issue is that this was only for a specific kind of task, that being summarization.

Ouyang et al. showed a framework that helped align language models with human intent in the paper Training language models to follow instructions with human feedback [3]. This paper essentially is what proved ChatGPT was viable. Some of the main limitations of chatbots can be seen, failure to deal with multiple constraints, failure to assertively respond to malicious queries with non-harmful responses. Furthermore, this approach still requires a very large amount of computational resources.

Christiano et al. developed an approach to apply human feedback on less than 1% of the interaction space, while remaining effective in the paper Deep Reinforcement Learning from Human Preferences [4]. This was a 2 part approach that trained a reward predictor based on human feedback, and then used that reward predictor to train a model to maximize the predicted reward. However, doing query generation one at a time is slow, so this is a fundamental limitation of this approach.

Bıyık et al. demonstrated an algorithm in their paper, Batch Active Preference-Based Learning of Reward Functions [5], which enables efficient learning of reward functions using as few data samples as possible. This approach generates queries in batches, solving the problem of the previous paper. While this did have a limitation of having a fixed batch size, it was much more practical than the single-query approach. This makes RLHF actually viable for robotics.

Lee et al., in their paper, PEBBLE: Feedback-Efficient Interactive Reinforcement Learning via Relabeling Experience and Unsupervised Pre-training [6], showed an off-policy approach that relabels agent's past experiences when its reward model changes. However, the paper does not discuss an interface for collecting human preference data, nor does it deal with the challenge of preventing potentially harmful or dangerous behaviors while learning. Our work is in large part based on PEBBLE, however we seek to integrate it into a larger system for using RLHF practically for robotics.

V. Methodology

A. Soft Actor Critic RL Algorithm

1) Entropy-Regularized Reinforcement Learning : Traditional reinforcement learning methods optimize policies by maximizing expected cumulative reward, which may lead to overly deterministic behaviors and poor exploration. Entropy-regularized reinforcement learning addresses this limitation by augmenting the objective with an entropy term that explicitly encourages stochasticity in the learned policy.

The entropy-regularized objective is defined as:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t\left(r(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))\right)\right], \quad (1)$$

where $\gamma \in (0,1)$ is the discount factor, $\alpha > 0$ is a temperature parameter, and $\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi}[\log\pi(a|s)]$ denotes the policy entropy. This formulation promotes robust exploration, which is critical for navigation in dynamic and uncertain environments.

2) SAC Bellman Function : Under the entropy-regularized framework, the soft Q-function for a policy π is defined as:

$$Q^\pi(s,a) = r(s,a) + \gamma\mathbb{E}_{s'\sim p}\left[V^\pi(s')\right], \quad (2)$$

where the soft value function is given by:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\left[Q^\pi(s,a) - \alpha\log\pi(a|s)\right]. \quad (3)$$

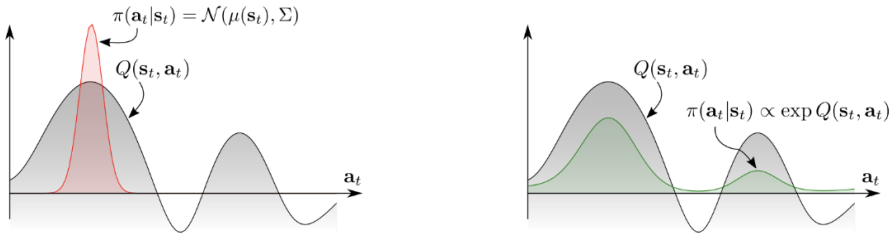


Fig. 2: The entropy term is added to approximate the policy network within a better distribution. The subfigure 3a is the standard RL algorithm without entropy terms; and the subfigure 3b is the entropy-based RL updated policy distribution.

The corresponding soft Bellman operator is a contraction mapping under standard assumptions on bounded rewards, ensuring convergence of soft policy evaluation through iterative updates.

3) Soft Actor-Critic *:* Soft Actor-Critic (SAC) is an off-policy, model-free reinforcement learning algorithm that optimizes the entropy-regularized objective. SAC maintains a stochastic policy (actor) and two soft Q-functions (critics) to mitigate overestimation bias.

The critics are trained by minimizing the soft Bellman residual:

$$\mathcal{L}(\phi_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}[(Q_{\phi_i}(s,a) - (r + \gamma \mathbb{E}_{a'\sim\pi}[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi(a'|s')]))^2] \quad (4)$$

The policy is updated by minimizing:

$$\mathcal{L}(\theta) = \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a|s) - \min_i Q_{\phi_i}(s,a)\right]. \quad (5)$$

This structure enables stable and sample-efficient learning, making SAC suitable for continuous control and socially-aware navigation tasks.

4) Soft Policy Evaluation proof: *:* More details refers to the Appendix. 20, 22, 23, 24, 21, 25.

For a fixed policy π, repeated application of the soft Bellman operator:

$$(\mathcal{T}^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[\mathbb{E}_{a'}\left[Q(s',a') - \alpha \log \pi(a'|s')\right]\right] \quad (6)$$

converges to the unique soft Q-function Q^π. This follows from the contraction property of $\mathcal{T}^\pi$ under the ℓ_∞ norm, ensuring stable policy evaluation.

5) Soft Policy Improvement proof: *:* More details refers to the Appendix. 20, 22, 23, 24, 21, 25.

Given a soft Q-function Q^π, a new policy π' is obtained by solving:

$$\pi' = \arg\max_\pi \mathbb{E}_{a\sim\pi}\left[Q^\pi(s,a) - \alpha \log \pi(a|s)\right]. \quad (7)$$

The solution is:

$$\pi'(a|s) \propto \exp\left(\frac{1}{\alpha} Q^\pi(s,a)\right). \quad (8)$$

This update guarantees:

$$Q^{\pi'}(s,a) \geq Q^\pi(s,a), \quad \forall s,a, \quad (9)$$

establishing monotonic improvement in the soft value function.

6) Soft Policy Iteration proof: *:* More details refers to the Appendix. 20, 22, 23, 24, 21, 25.

Soft policy iteration alternates between soft policy evaluation and soft policy improvement. Since evaluation converges to Q^π and improvement produces a policy with non-decreasing soft value, the sequence of policies converges to an optimal maximum-entropy policy π^*.

This theoretical guarantee underpins the convergence properties of Soft Actor-Critic and motivates its selection as the optimization backbone in the proposed system.

B. Reinforcement Learning from Human Feedback Algorithm

Reinforcement Learning from Human Feedback (RLHF) replaces or augments manually designed reward functions with a reward model learned from preference-based feedback. [3], [6] Rather than assigning scalar rewards to individual state–action pairs, a supervisor provides comparisons between short trajectory segments, indicating which behavior is preferred. The RLHF framework used in this project consists of three components: a preference model, a maximum-likelihood loss for reward learning, and policy optimization using the learned reward.

1) Bradley-Terry Preference Model *:* Let τ_A and τ_B denote two trajectory segments, where each trajectory is a sequence of state–action pairs. The reward model $r_\psi(s,a)$ assigns a scalar reward to each state–action pair, and the total score of a trajectory is defined as the sum of predicted rewards along the trajectory:

$$R_\psi(\tau) = \sum_t r_\psi(s_t, a_t).$$

Following the Bradley-Terry preference model [6], [7], the probability that trajectory τ_A is preferred over τ_B is given by:

$$P(\tau_A \succ \tau_B) = \frac{\exp\left(R_\psi(\tau_A)\right)}{\exp\left(R_\psi(\tau_A)\right) + \exp\left(R_\psi(\tau_B)\right)}.$$

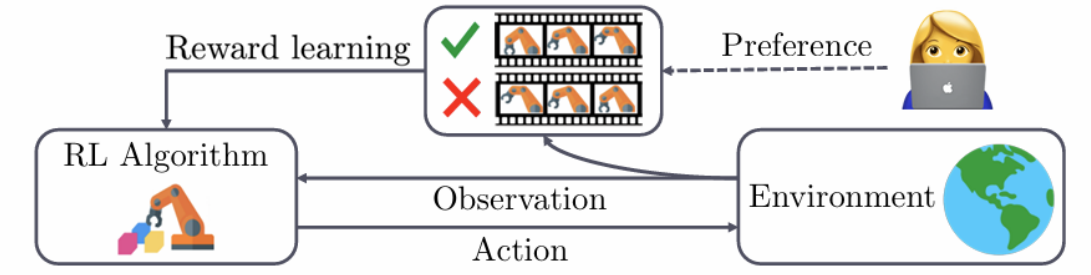


Fig. 3: Illustration of preference-based reinforcement learning. Instead of assuming access to a hand-engineered reward function, a supervisor provides preferences between pairs of agent behaviors. These preferences are used to learn a reward model that guides policy optimization.

This formulation ensures that trajectories with higher predicted cumulative reward are more likely to be preferred, while still allowing for uncertainty in the preference signal.

2) Maximum Likelihood Estimation Loss : Given a dataset of preference comparisons $\{(\tau_A, \tau_B, y)\}$, where $y = 1$ indicates that τ_A is preferred over τ_B and $y = 0$ indicates the opposite, the reward model is trained via maximum likelihood estimation. [6] The corresponding loss function is:

$$\mathcal{L}_{\text{pref}}(\psi) = -\mathbb{E}\left[y \log P(\tau_A \succ \tau_B) + (1-y) \log P(\tau_B \succ \tau_A)\right].$$

Minimizing this loss encourages the reward model to assign higher cumulative rewards to trajectories that are consistently preferred by the supervisor.

In this project, preference labels are generated automatically by comparing the true environment returns of trajectory segments, serving as a simulated human supervisor. While this limits the expressiveness of the feedback, it enables controlled comparison between standard reinforcement learning and RLHF under identical conditions.

3) Optimization of the Reward Model and Policy : The reward model is trained offline using collected preference data and then used to replace the original environment reward during policy learning. Once the reward model converges, the policy is optimized using the Soft Actor-Critic (SAC) algorithm with the learned reward: [8]. All the parameter in the SAC can be accessed in Fig. 26.

$$\mathbb{E}_\pi \left[\sum_t r_\psi(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right].$$

This two-stage optimization decouples reward specification from policy learning, allowing the agent to optimize behavior aligned with preference-based feedback rather than explicitly designed rewards. [6] Although simulated preferences constrain the potential performance gains of RLHF in this setting, the framework provides a principled way to study the effect of preference-based supervision across multiple robotic environments.

To furthermore approximate the noise distribution of human supervisors, the stochastic preference predictor in our RLHF algorithm is defined as follows:

$$P\left[\sigma^{i_1} \succ \sigma^{i_2}; \beta, \gamma\right] = \frac{\exp\left(\beta \sum_{t=1}^{H} \gamma^{H-t} R\left(s_t^{i_1}, a_t^{i_1}\right)\right)}{\sum_{i \in \{i_1, i_2\}} \left[\exp\left(\beta \sum_{t=1}^{H} \gamma^{H-t} R\left(s_t^{i}, a_t^{i}\right)\right)\right]}.$$

where λ denotes the discounted parameter, and β is the rationality constant. All the parameter in the RLHF can be accessed in Fig. 29.

VI. Experiment

A. Environmental Configuration

We conduct the experiments over several robotics environments from Gymnasium.

1) Humanoid Stand-up: A 3D bipedal robot is initialized laying on the ground, the goal of the environment is to stand up and remain standing by applying various torques to the appendages.
2) Humanoid Walk: A 3D bipedal robot is initialized in a standing position, the goal of the environment is to walk forward as quickly as possible.
3) Walker2D Stand-up: A 2D bipedal robot is initialized laying on the ground, the goal of the environment is to stand up and remain standing by applying various torques to the appendages.
4) Walker2D Walk: A 2D bipedal robot is initialized in a standing position, the goal of the environment is to walk forward as quickly as possible.
5) Quadruped Ant Walk: A 3D quadruped robot is initialized in a standing position, the goal of the environment is to walk forward as quickly as possible.
6) Social Robot Navigation: A 2D robot is initialized such that it can move freely in the 2D space. The goal of the environment is to navigate around and conveniently avoid other agents within the environment, according to social standards.

B. Results

Our experiments were conducted to evaluate the performance of existing SOTA RL algorithm SAC [8] and RLHF algorithm [6] on a set of above environments. Each environment has two distinct random seeds for both SAC and RLHF policy.

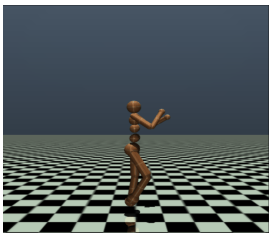


Fig. 4: Humanoid ENV

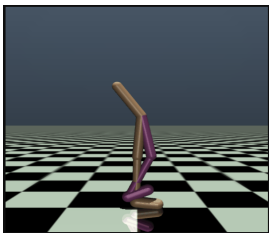


Fig. 5: Walker ENV

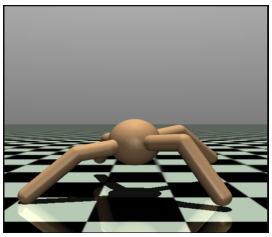


Fig. 6: Ant ENV

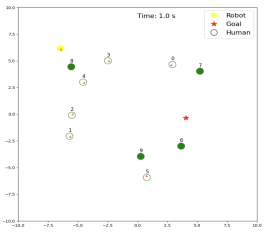


Fig. 7: Navigation ENV

1) Walker2D-Walk: Evaluation Performance: Figure 8 reports the evaluation episode reward of SAC and RLHF on Walker2D-Walk as training progresses. Both methods eventually reach high returns, but SAC attains near-optimal performance much faster and converges to slightly higher final rewards, indicating more sample-efficient learning under the true environment reward. RLHF learns more slowly and plateaus at a lower asymptotic return, which is expected because the policy optimizes a learned preference-based reward model rather than the ground-truth task reward [3], [6]. 1. SAC and RLHF for Walker2D-Walk

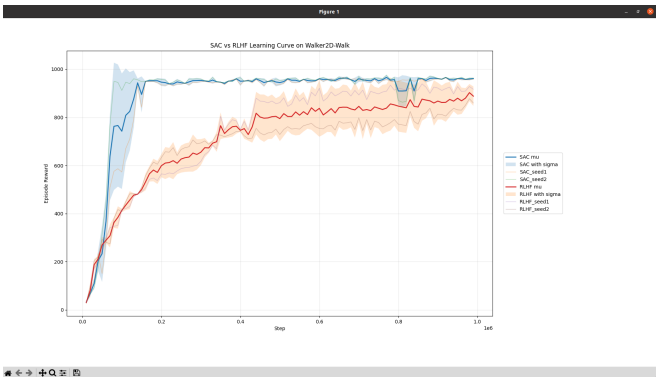


Fig. 8: The evaluation metric of testing procedure episode reward for SAC and RLHF algorithm on Walker2D-Walk environment.

2) Walker2D-Walk: Training Episode Reward: The second plot in Figure 9 shows the training episode reward, i.e., the return collected by the agents on-policy during learning. SAC again exhibits a steep performance increase in the early stages, followed by stable high rewards with occasional drops due to exploration. In contrast, RLHF improves more gradually over the entire 10^6-step horizon and never matches the SAC training return, reflecting both the additional noise introduced by the learned reward model and the fact that the simulated preference labels only approximate the environment returns [6]. This behavior is consistent with prior findings that preference-based RL typically trades off some peak performance for the ability to optimize non-engineered objectives [3], [6].

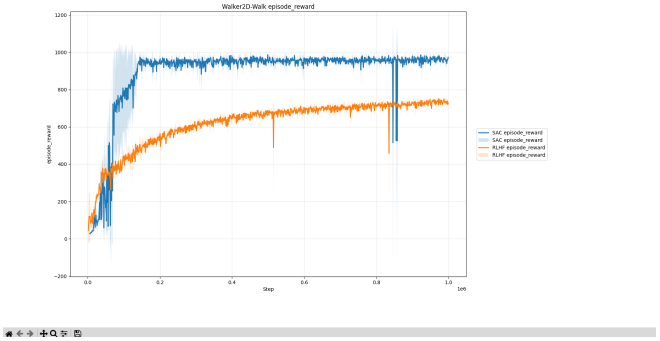


Fig. 9: The evaluation metric of training procedure episode reward for SAC and RLHF algorithm on Walker2D-Walk environment.

3) Walker2D-Walk: Actor Entropy: The third plot in Figure 10 tracks the actor entropy for both algorithms. For SAC, the entropy starts high and quickly decays toward the target entropy as the policy concentrates on rewarding actions, which is characteristic of maximum-entropy RL with an automatically tuned temperature [8]. RLHF shows a very similar entropy schedule, dropping from an initially exploratory policy to a near-deterministic policy early in training, which suggests that the RLHF agent is not maintaining higher exploration than SAC despite being trained on a learned reward. This alignment of entropy curves indicates that the main differences between SAC and RLHF on Walker2D-Walk arise from reward modeling rather than from qualitatively different exploration behavior [6].

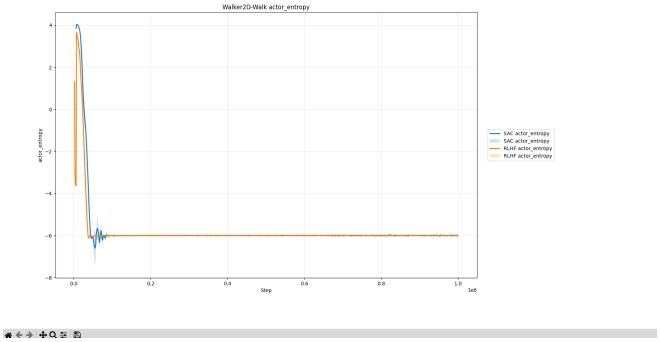


Fig. 10: The evaluation metric of actor entropy for SAC and RLHF algorithm on Walker2D-Walk environment.

4) Walker2D-Walk: Actor Loss: Figure 11 reports the actor loss for SAC and RLHF over training steps on Walker2D-

Walk. For both methods the actor loss quickly decreases from its initial value, indicating that the policies rapidly move toward regions of the action space that yield higher expected returns under their respective objectives [8]. SAC's actor loss decays more steeply and stabilizes at a lower magnitude than RLHF, which is consistent with its faster convergence and higher final episode rewards; the slower, higher trajectory of the RLHF actor loss reflects the added noise from optimizing a learned preference-based reward model instead of the ground-truth task reward [3], [6].

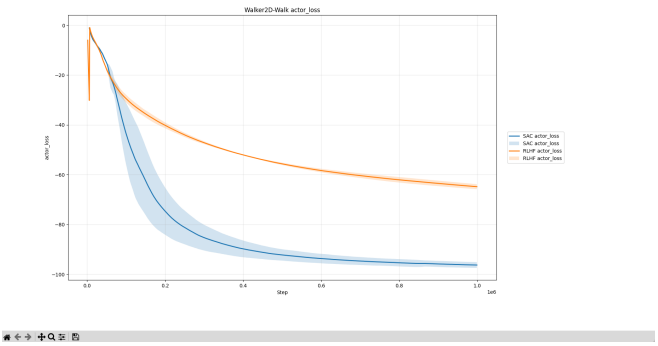


Fig. 11: The evaluation metric of actor loss for SAC and RLHF algorithm on Walker2D-Walk environment.

5) Walker2D-Walk: Critic Entropy: Figure 12 shows the critic entropy for RLHF as training progresses. The entropy starts high at the beginning of training, when the value function is still highly uncertain about returns, and then rapidly collapses toward zero, indicating that the critic quickly becomes confident in its predictions under the learned reward model. After this initial transient, the entropy curve remains essentially flat, suggesting that most of the critic's uncertainty is resolved early and that later updates mainly refine a nearly deterministic value estimate rather than exploring alternative hypotheses about returns [6], [8].

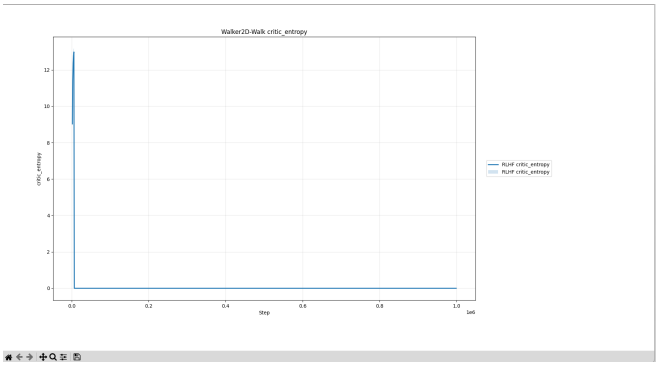


Fig. 12: The evaluation metric of critic entropy for SAC and RLHF algorithm on Walker2D-Walk environment.

6) Walker2D-Walk: Critic Loss: Figure 13 presents the critic loss for SAC and RLHF. Both critics exhibit an early spike in loss followed by a gradual decay, which corresponds to the transition from a randomly initialized Q-function to one that fits the observed returns for the evolving policy. Over time, SAC reaches a lower critic loss than RLHF, indicating a more accurate approximation of the soft Q-values under the true environment reward, whereas the RLHF critic retains a slightly higher residual error because it must fit values under a non-stationary, learned reward function that is updated from preference data [3], [6].

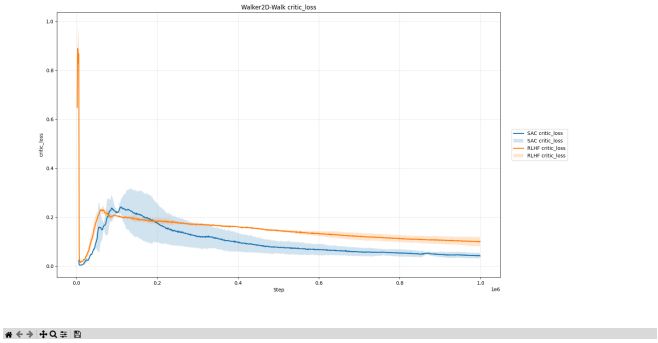


Fig. 13: The evaluation metric of critic loss for SAC and RLHF algorithm on Walker2D-Walk environment.

2. The comparison experiments of SAC and RLHF for Walker2D-Standup, Humanoid-Walk, Humanoid-Standup, Ant-Walk can be found in the Appendix part.

7) Walker2D-Standup: Figure 14 shows the evaluation episode reward of SAC and RLHF on Walker2D-Standup. SAC rapidly improves and reaches near-maximal return early in training, while RLHF climbs more slowly and saturates at a lower reward, mirroring the Walker2D-Walk results and indicating that optimizing the learned preference-based reward sacrifices some asymptotic performance for this task [6], [8].

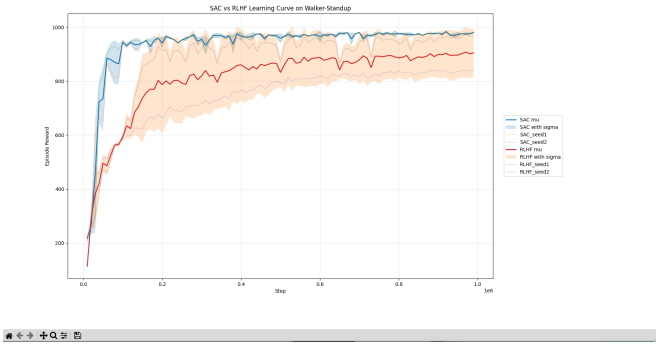


Fig. 14: The evaluation metric of testing procedure episode reward for SAC and RLHF algorithm on Walker2D-Standup environment.

8) Humanoid-Standup: In Figure 15, SAC steadily improves throughout training on Humanoid-Standup and achieves a substantial positive return, whereas RLHF remains close to zero reward and shows almost no learning signal. This suggests that, under the current preference-generation scheme and reward-model capacity, RLHF fails to provide a

useful shaping signal for the much more complex humanoid standup behavior, highlighting a limitation of directly transferring the same RLHF pipeline across tasks of very different difficulty [3], [6].

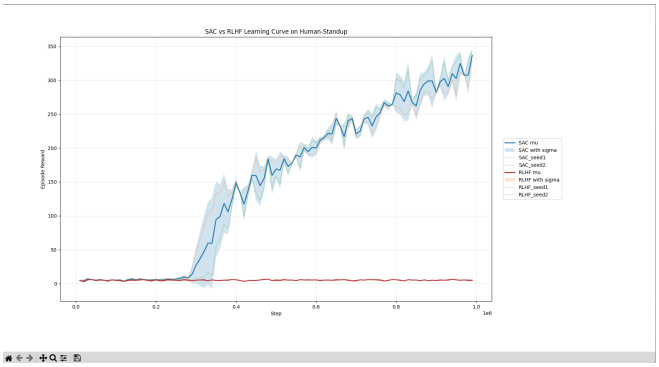


Fig. 15: The evaluation metric of testing procedure episode reward for SAC and RLHF algorithm on Humanoid-Standup environment.

9) Humanoid-Walk: Figure 16 presents results on Humanoid-Walk. SAC again exhibits clear learning, with episode reward increasing over time and displaying some variance across seeds as the gait stabilizes. In contrast, RLHF rewards stay near zero across the full training horizon, indicating that the agent does not discover walking behavior when guided only by the learned preference-based reward on this high-dimensional control problem.

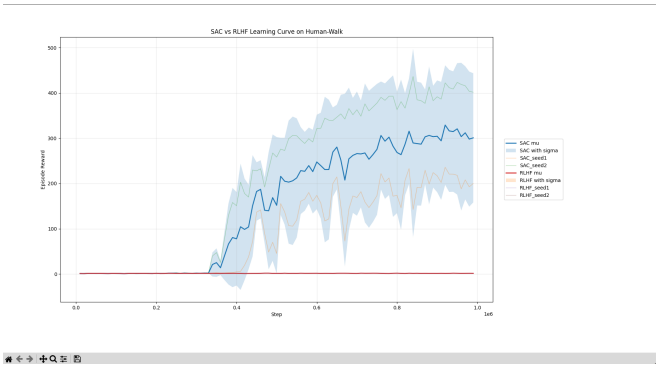


Fig. 16: The evaluation metric of testing procedure episode reward for SAC and RLHF algorithm on Humanoid-Walk environment.

10) Ant-Walk: Figure 17 reports the learning curves for Ant-Walk. SAC gradually improves from low initial returns to high final rewards with moderate variance, demonstrating successful locomotion learning. RLHF does learn to increase reward somewhat over time, but its performance remains far below SAC and much noisier, showing that while the preference-based reward is informative enough to encourage some forward progress, it is significantly weaker than the true environment reward for driving efficient ant locomotion [6], [8].

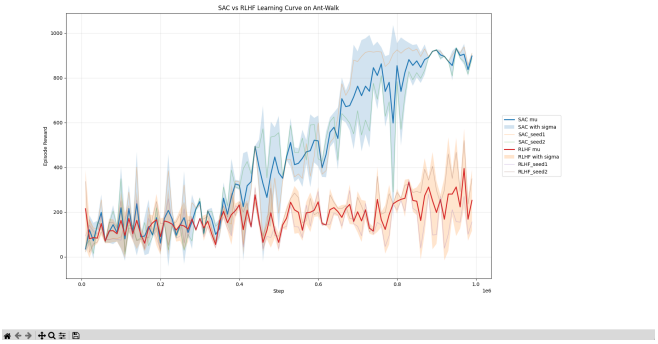


Fig. 17: The evaluation metric of testing procedure episode reward for SAC and RLHF algorithm on Ant-Walk environment.

VII. Conclusion and Future Work

We integrated preference-based RLHF (Bradley Terry reward modeling) with an off-policy SAC backbone and evaluated it on locomotion and navigation tasks. RLHF often yields a smoother, more informative learning signal, improving reward learning efficiency once supervision is consistent. However, overall task performance is bounded by feedback quality: with a scripted supervisor, the learned reward can be misaligned on hard tasks (e.g., Humanoid-Walk), while remaining competitive with SAC on easier tasks (e.g., Walker-Standup) where the proxy aligns better with success.

Apart from that, the future work includes: Replace scripted supervision with real human preference labels and measure label efficiency and noise tolerance; Improve reward-model robustness (ensembles/uncertainty, drift detection, periodic relabeling); Use active querying and hybrid signals (preferences and safety/auxiliary objectives) to reduce feedback cost; Validate on real robots with safety constraints and broader metrics beyond return (stability, smoothness, safety, social compliance).

ACKNOWLEDGMENT

This is a course project final report for ECE-59500 Reinforcement Learning at Purdue University, under the supervision of Prof. Ghasemi.

References

[1] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, "Fine-tuning language models from human preferences," *arXiv preprint arXiv:1909.08593*, 2019. [Online]. Available: https://arxiv.org/abs/1909.08593

[2] N. Stiennon, L. Ouyang, J. Wu, D. M. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. Christiano, "Learning to summarize from human feedback," *arXiv preprint arXiv:2009.01325*, 2020. [Online]. Available: https://arxiv.org/abs/2009.01325

[3] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[4] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," 2023. [Online]. Available: https://arxiv.org/abs/1706.03741

[5] E. Bıyık and D. Sadigh, "Batch active preference-based learning of reward functions," *arXiv preprint arXiv:1810.04303*, 2018. [Online]. Available: https://arxiv.org/abs/1810.04303

[6] K. Lee, L. Smith, and P. Abbeel, "Pebble: Feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training," *International Conference on Machine Learning*, 2021.

[7] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. the method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.

[8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," 2018. [Online]. Available: https://arxiv.org/abs/1801.01290

APPENDIX

Note: All the experiment program and result files can be accessed via the BrightSpace or GradeSpace.

A. 1. SAC Theoretical Proof

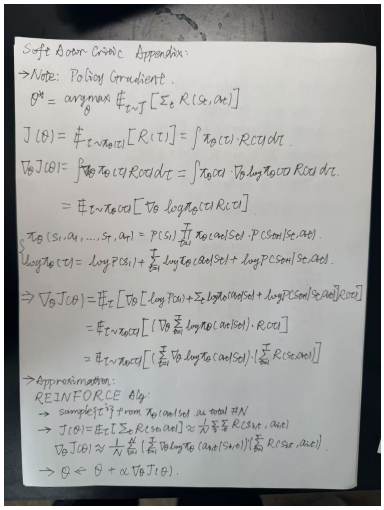


Fig. 18: Fundamental Definitions of Policy Gradients and Actor-Critic Methods

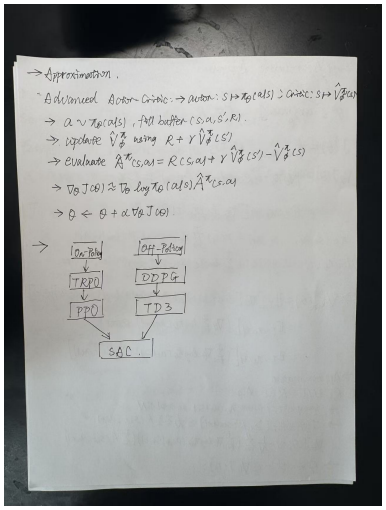


Fig. 19: Fundamental Definitions of Policy Gradients and Actor-Critic Methods

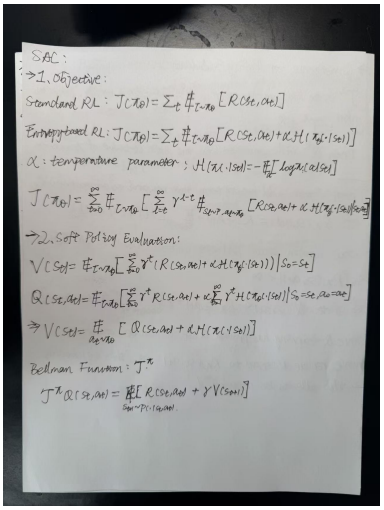


Fig. 20: Definition of SAC objectives

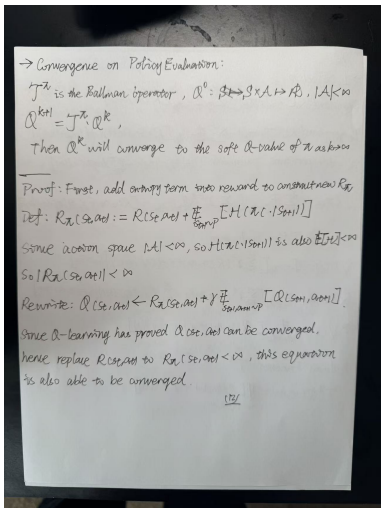


Fig. 21: Convergence of Policy Evaluation

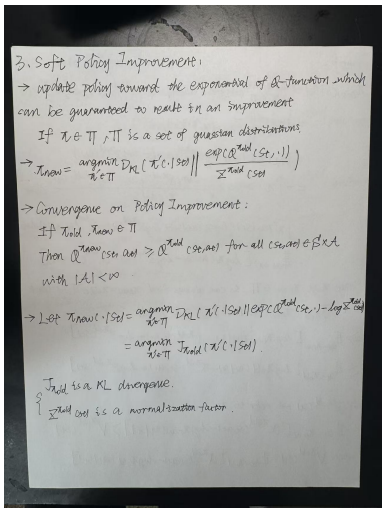


Fig. 22: Convergence of Policy Improvement

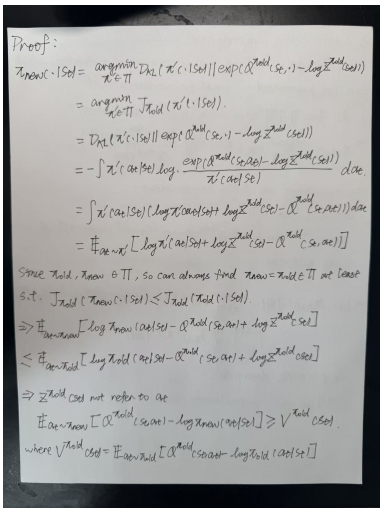


Fig. 23: Convergence of Policy Improvement

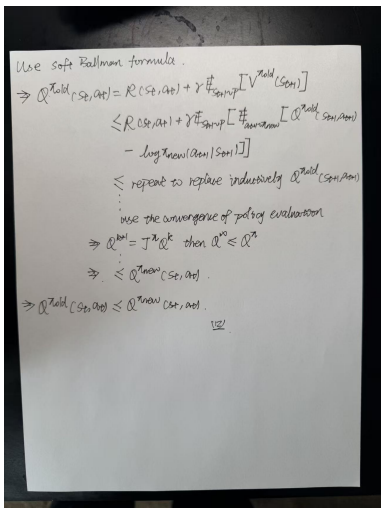


Fig. 24: Convergence of Policy Improvement

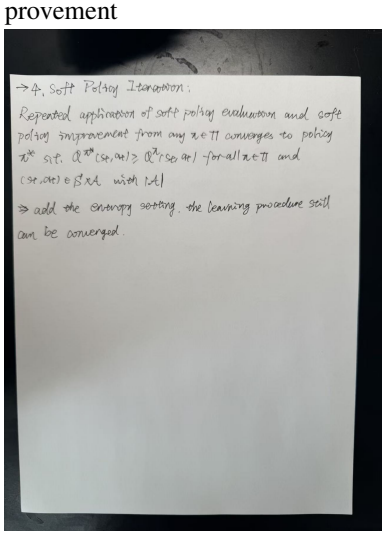


Fig. 25: Convergence of SAC Learning Procedure

B. 2. SAC and RLHF for Walker2D-walk

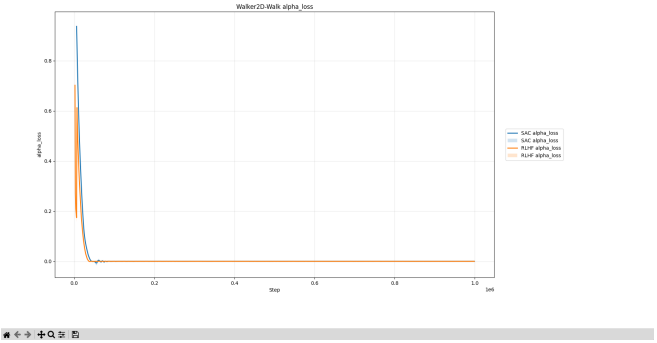


Fig. 33: The evaluation metric of alpha loss for SAC and RLHF algorithm on Walker2D-Walk environment.

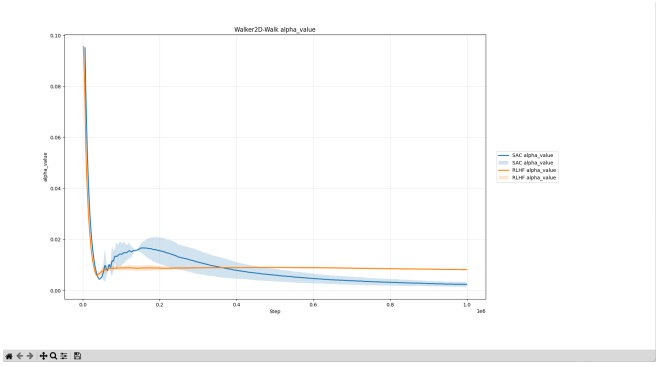


Fig. 34: The evaluation metric of alpha value for SAC and RLHF algorithm on Walker2D-Walk environment.

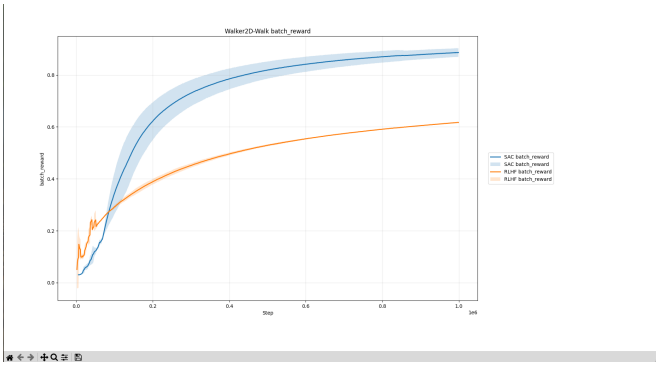


Fig. 35: The evaluation metric of batch reward for SAC and RLHF algorithm on Walker2D-Walk environment.

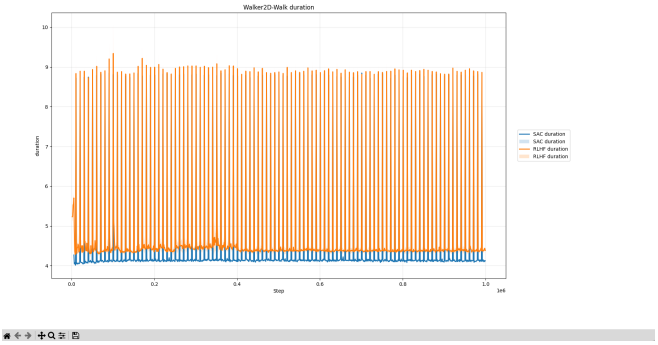


Fig. 36: The evaluation metric of duration for SAC and RLHF algorithm on Walker2D-Walk environment.

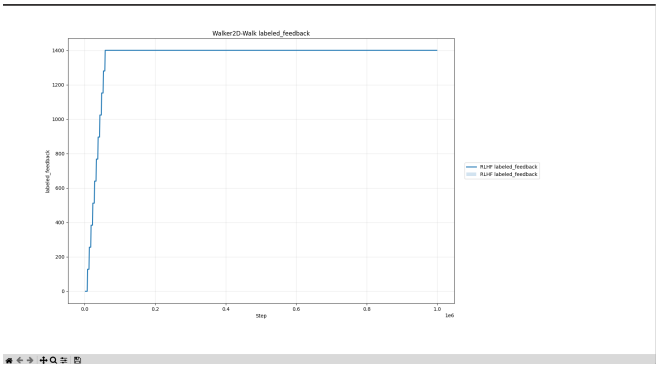


Fig. 37: The evaluation metric of labeled feedback for SAC and RLHF algorithm on Walker2D-Walk environment.

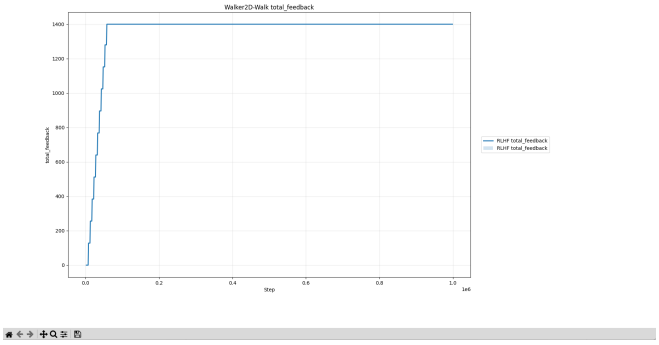


Fig. 38: The evaluation metric of total feedback for SAC and RLHF algorithm on Walker2D-Walk environment.

```yaml
agent:
  class: agent.sac.SACAgent
  name: sac
  params:
    action_dim: ???
    action_range: ???
    actor_betas:
    - 0.9
    - 0.999
    actor_cfg: ${diag_gaussian_actor}
    actor_lr: 0.0001
    actor_update_frequency: 1
    alpha_betas:
    - 0.9
    - 0.999
    alpha_lr: 0.0001
    batch_size: 1024
    critic_betas:
    - 0.9
    - 0.999
    critic_cfg: ${double_q_critic}
    critic_lr: 0.0001
    critic_target_update_frequency: 2
    critic_tau: 0.005
    device: ${device}
    discount: 0.99
    init_temperature: 0.1
    learnable_temperature: true
    obs_dim: ???
device: cuda
diag_gaussian_actor:
  class: agent.actor.DiagGaussianActor
  params:
    action_dim: ${agent.params.action_dim}
    hidden_depth: 2
    hidden_dim: 1024
    log_std_bounds:
    - -5
    - 2
    obs_dim: ${agent.params.obs_dim}
double_q_critic:
  class: agent.critic.DoubleQCritic
  params:
    action_dim: ${agent.params.action_dim}
    hidden_depth: 2
    hidden_dim: 1024
    obs_dim: ${agent.params.obs_dim}
env: walker_walk
eval_frequency: 10000
experiment: sac
log_frequency: 10000
log_save_tb: true
num_eval_episodes: 10
num_seed_steps: 5000
num_train_steps: 1000000.0
num_unsup_steps: 0
replay_buffer_capacity: ${num_train_steps}
reset_update: 100
save_model: true
save_video: false
seed: 2
topK: 5
```

Fig. 26: SAC algorithm parameters configuration.

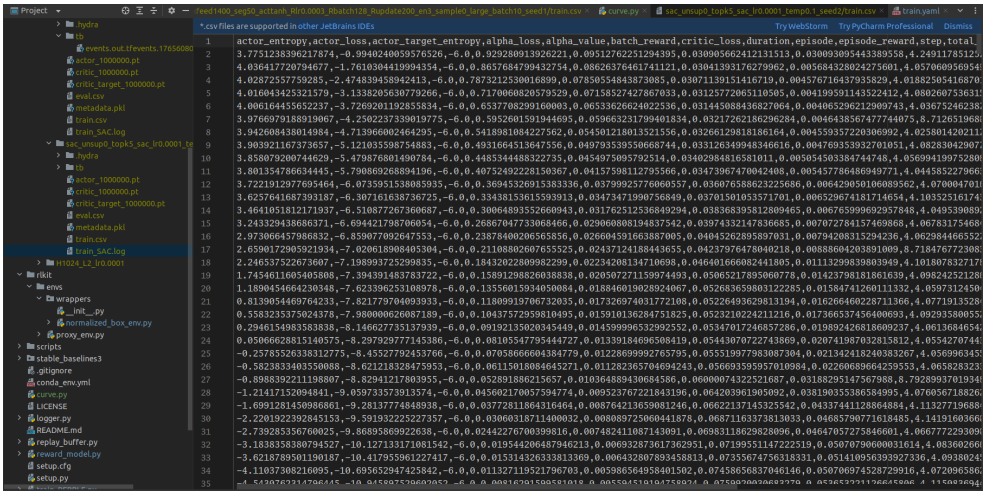
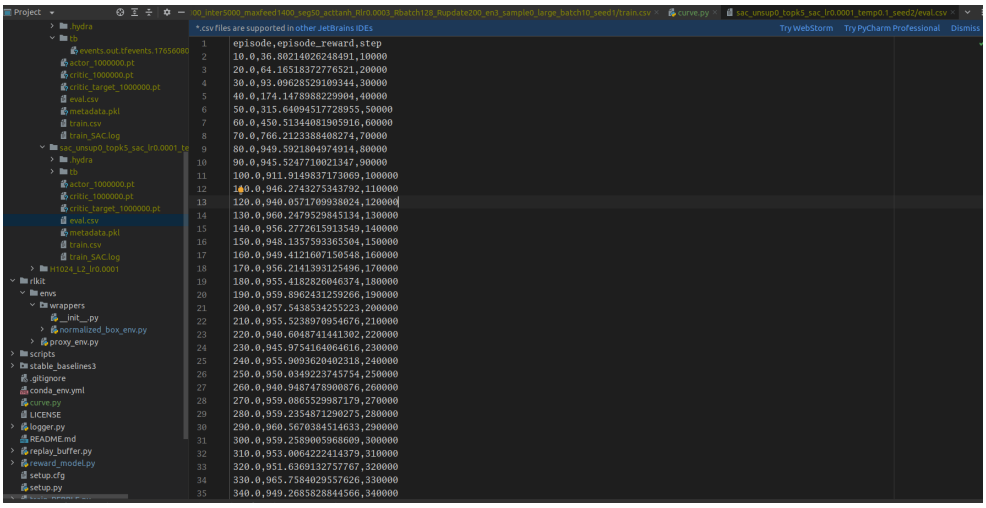


Fig. 27: SAC training log example.

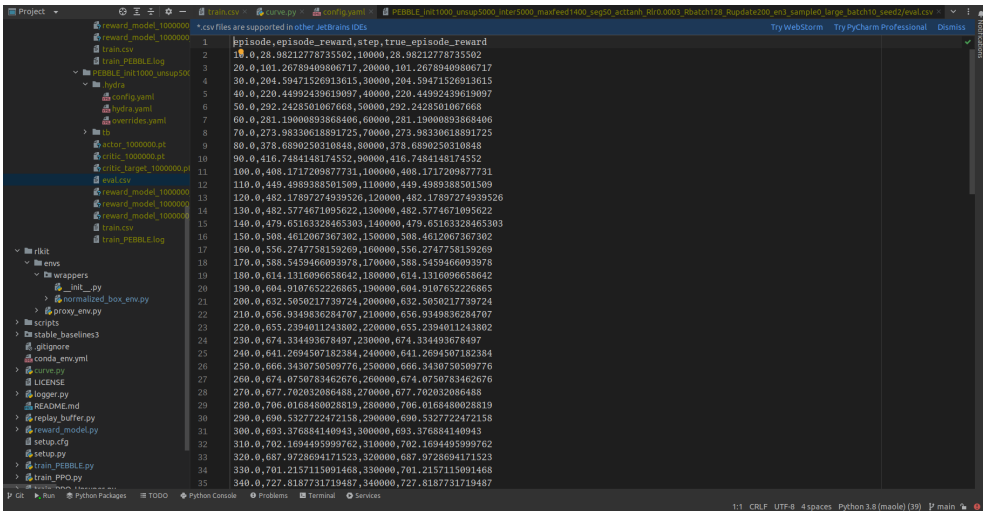


```
episode,episode_reward,step
10.0,36.80214026248491,10000
20.0,64.16518372776521,20000
30.0,93.09628529109344,30000
40.0,174.1478988229904,40000
50.0,315.6409451772895,50000
60.0,450.5134408190591,60000
70.0,766.2123388408274,70000
80.0,949.5921804974914,80000
90.0,945.5247710021347,90000
100.0,911.9149837173069,100000
110.0,946.2743275343792,110000
120.0,940.0571709938024,120000
130.0,960.2479529845134,130000
140.0,956.2772615913549,140000
150.0,948.1357593365504,150000
160.0,949.4121607150548,160000
170.0,956.2141393125496,170000
180.0,955.4182826046374,180000
190.0,959.8962431259266,190000
200.0,957.5438534255223,200000
210.0,955.5238970954676,210000
220.0,940.6048741441302,220000
230.0,945.9754164064616,230000
240.0,955.9093620402318,240000
250.0,950.0349223745754,250000
260.0,940.9487478900876,260000
270.0,959.0865529987179,270000
280.0,959.2354871290275,280000
290.0,960.5670384514633,290000
300.0,959.2589005968609,300000
310.0,953.0064222414379,310000
320.0,951.6369132757767,320000
330.0,965.7584029557626,330000
340.0,949.2685828844566,340000
```

Fig. 28: SAC evaluation log example.

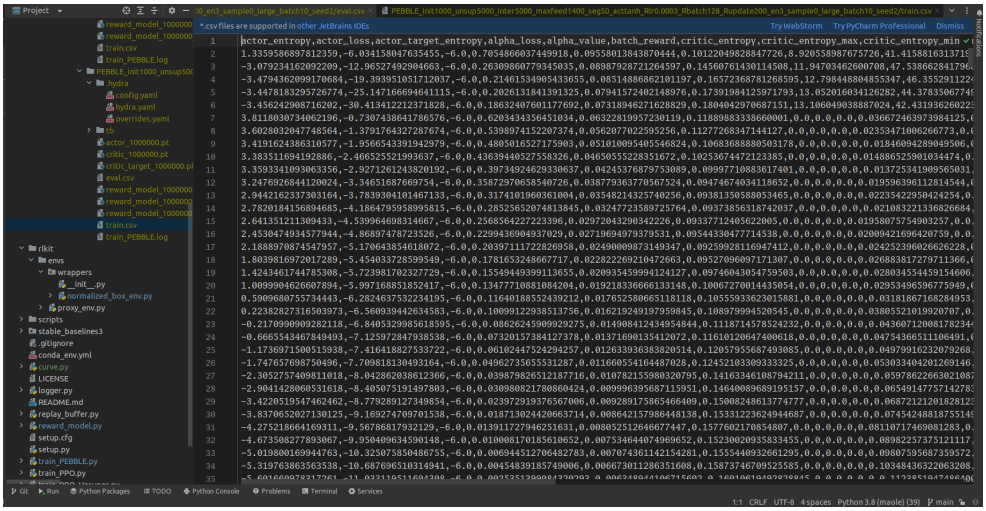
```
activation: tanh
agent:
  class: agent.sac.SACAgent
  name: sac
  params:
    action_dim: ???
    action_range: ???
    actor_betas:
    - 0.9
    - 0.999
    actor_cfg: ${diag_gaussian_actor}
    actor_lr: 0.0001
    actor_update_frequency: 1
    alpha_betas:
    - 0.9
    - 0.999
    alpha_lr: 0.0001
    batch_size: 1024
    critic_betas:
    - 0.9
    - 0.999
    critic_cfg: ${double_q_critic}
    critic_lr: 0.0001
    critic_target_update_frequency: 2
    critic_tau: 0.005
    device: ${device}
    discount: 0.99
    init_temperature: 0.1
    learnable_temperature: true
    obs_dim: ???
device: cuda
diag_gaussian_actor:
  class: agent.actor.DiagGaussianActor
  params:
    action_dim: ${agent.params.action_dim}
    hidden_depth: 2
    hidden_dim: 1024
    log_std_bounds:
    - -5
    - 2
    obs_dim: ${agent.params.obs_dim}
double_q_critic:
  class: agent.critic.DoubleQCritic
  params:
    action_dim: ${agent.params.action_dim}
    hidden_depth: 2
    hidden_dim: 1024
    obs_dim: ${agent.params.obs_dim}
ensemble_size: 3
env: walker_walk
eval_frequency: 10000
experiment: PEBBLE
feed_type: 0
gradient_update: 1
label_margin: 0.0
large_batch: 10
log_frequency: 10000
log_save_tb: true
max_feedback: 1400
num_eval_episodes: 10
num_interact: 5000
num_seed_steps: 1000
num_train_steps: 1000000.0
num_unsup_steps: 5000
replay_buffer_capacity: ${num_train_steps}
reset_update: 100
reward_batch: 128
reward_lr: 0.0003
reward_schedule: 0
reward_update: 200
save_video: false
seed: 2
segment: 50
teacher_beta: -1
teacher_eps_equal: 0
teacher_eps_mistake: 0
teacher_eps_skip: 0
teacher_gamma: 1
topK: 5
```

Fig. 29: RLHF algorithm parameters configuration.



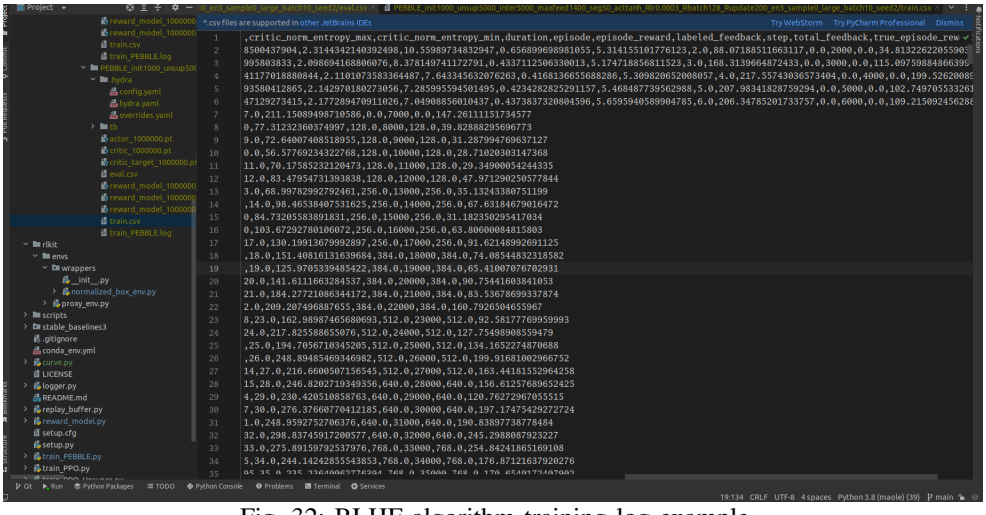
```
episode,episode_reward,step,true_episode_reward
10.0,28.982127778735502,10000,28.982127778735502
20.0,101.26789409806717,20000,101.26789409806717
30.0,204.59471526913615,30000,204.59471526913615
40.0,220.44992439619097,40000,220.44992439619097
50.0,292.2428501067668,50000,292.2428501067668
60.0,281.19000893868406,60000,281.19000893868406
70.0,273.98330618891725,70000,273.98330618891725
80.0,378.6890250310848,80000,378.6890250310848
90.0,416.7484148174552,90000,416.7484148174552
100.0,408.1717209877731,100000,408.1717209877731
110.0,449.4989388501509,110000,449.4989388501509
120.0,482.17897274939526,120000,482.17897274939526
130.0,482.5774671095622,130000,482.5774671095622
140.0,479.65163328465303,140000,479.65163328465303
150.0,508.4612067367302,150000,508.4612067367302
160.0,556.2747758159269,160000,556.2747758159269
170.0,588.5459466093978,170000,588.5459466093978
180.0,614.1316096658642,180000,614.1316096658642
190.0,604.9107652226865,190000,604.9107652226865
200.0,632.5050217739724,200000,632.5050217739724
210.0,656.9349836284707,210000,656.9349836284707
220.0,655.2394011243802,220000,655.2394011243802
230.0,674.334493678497,230000,674.334493678497
240.0,641.2694507182384,240000,641.2694507182384
250.0,666.3430750509776,250000,666.3430750509776
260.0,674.0750783462676,260000,674.0750783462676
270.0,677.702032086488,270000,677.702032086488
280.0,706.0168480028819,280000,706.0168480028819
290.0,690.5327724272158,290000,690.5327724272158
300.0,693.3768841409 43,300000,693.3768841409043
310.0,702.1694495999762,310000,702.1694495999762
320.0,687.9728694171523,320000,687.9728694171523
330.0,701.2157115091468,330000,701.2157115091468
340.0,727.8187731719487,340000,727.8187731719487
```

Fig. 30: RLHF algorithm evaluation log example.



```
actor_entropy,actor_loss,actor_target_entropy,alpha_loss,alpha_value,batch_reward,critic_entropy,critic_entropy_max,critic_entropy_min
1.3359586897812359,-6.034158047635455,-6.0,0.7054866037449918,0.9955801384870444,0.10122049828847726,8.920558987675726,41.41588163137157
-3.079234162092209,-12.965274929044663,-6.0,0.2630986077934503,0.08987928721264597,0.14560761430114508,11.947034626007084,47.538662841796
-3.4794362099170684,-19.393951051712037,-6.0,0.21461534905433655,0.08514886862101197,0.16572368781268595,12.798448804855347,46.3552911224
-3.4478183295726774,-25.147166694641115,-6.0,0.2026131841391325,0.07941572402148976,0.17391984125971793,13.052016034126282,44.37835067749
-3.456242908716202,-30.413412212371828,-6.0,0.18632407601177692,0.07318946271628829,0.18040429706871151,13.106049388887024,42.431936260223
3.811803073406219 6,-0.7307438641786576,-6.0,0.6203434356451034,0.06322819957230119,0.11889883338660001,0.0,0.0,0.0,0.0,0.03667246397398412 5,
3.602803204774854 6,-1.3791764327287674,-6.0,0.5398974152207374,0.05620770225952566,0.11277268347144127,0.0,0.0,0.0,0.0,0.02353471006266773,0.0,
3.4191624386310577,-1.9566543391942979,-6.0,0.4805016527175903,0.05101009540556462 4,0.10683688880503178,0.0,0.0,0.0,0.0,0.01846094289049506,
3.3835116941928866,-2.4665255521993637,-6.0,0.4363944052755832 6,0.04650555228351672,0.10253674472123385,0.0,0.0,0.0,0.0,0.01488652590103447 4,0.
3.3593341093063356,-2.9271261243820192,-6.0,0.3973492462933063 7,0.04245376879753089,0.09997710883617401,0.0,0.0,0.0,0.0,0.01372534190956031,
3.2476926844120024,-3.3465168766975 4,-6.0,0.3587297065854726,0.038779363770567524,0.09474674034118652,0.0,0.0,0.0,0.0,0.01959639611281454 4,
2.9442162337303164,-3.7839304101467133,-6.0,0.3174101960361004,0.03548214325740256,0.09381350588053465,0.0,0.0,0.0,0.0,0.02235422950424254,0.
2.7820184156894685,-4.186479593895315,-6.0,0.2852563207481384 5,0.03247723589725764,0.09373856318742037,0.0,0.0,0.0,0.0,0.02108322136826684,
2.6413512113094 33,-4.539964698314667,-6.0,0.2568564227223396,0.02972043290342226,0.09337712405622005,0.0,0.0,0.0,0.0,0.01958075749032 57,0.0,
2.4530474934577944,-4.86897478723526,-6.0,0.2299436904937029,0.027196949779379531,0.09544330477714538,0.0,0.0,0.0,0.0,0.02009421696420759,0.0,
2.1888970874547957,-5.170643854618072,-6.0,0.2039711172282695 8,0.02490009873149347,0.09259928116947412,0.0,0.0,0.0,0.0,0.02425239602662622 8,
1.8039816972017289,-5.454033728599549,-6.0,0.1781653248667717,0.022822269210472663,0.09527096097171307,0.0,0.0,0.0,0.0,0.02688381727971136 6,
1.4243461744785308,-5.723981702327279,-6.0,0.1554944939113655,0.020935459994124127,0.09746043054759503,0.0,0.0,0.0,0.0,0.02803455459154606,
1.0099904626607894,-5.997168851852417,-6.0,0.1347771081084204,0.019218336666133148,0.10067270014435054,0.0,0.0,0.0,0.0,0.02953496596775949,
0.5909680755734443,-6.282463753223 95,-6.0,0.1164018855249321 2,0.01765280665118118,0.10555933623015881,0.0,0.0,0.0,0.0,0.03181867168284953,
0.2238282731650397 3,-6.560939442634583,-6.0,0.1009912938531756,0.016219249197959845,0.10897999452054 5,0.0,0.0,0.0,0.0,0.03805521019920707,0.
-0.2170990909282118,-6.840532998618595,-6.0,0.0862624590992927 5,0.01490841243495484 4,0.11187145785242 32,0.0,0.0,0.0,0.0,0.04360712008178234
-0.6665543467849493,-7.125972847938538,-6.0,0.0732015738412737 8,0.013716901354120 72,0.11610120674000618,0.0,0.0,0.0,0.0,0.04754366511106491,
-1.1736971500515938,-7.416418827533722,-6.0,0.0610244752429425 7,0.01263393638382051 4,0.12057955687493085,0.0,0.0,0.0,0.0,0.04979916232079268,
-1.7476576986750496,-7.709818130493164,-6.0,0.0496273565531287,0.011660554164487028,0.12452103039333325,0.0,0.0,0.0,0.0,0.05303340420126914 6,
-2.3052757409811018,-8.042862038612366,-6.0,0.0398782651218771 6,0.010782155980320795,0.14163346108794211,0.0,0.0,0.0,0.0,0.0597862266302108 7,
-2.9041428060531618,-8.405075191497803,-6.0,0.0309808217806042 4,0.009996395687115951,0.14640896891955157,0.0,0.0,0.0,0.0,0.06549147757142785
-3.4220519547462462,-8.779289127349854,-6.0,0.0239729193765670 06,0.009289175865466409,0.15008248613774777,0.0,0.0,0.0,0.0,0.06872121201828123
-3.8370652027130125,-9.169274709701538,-6.0,0.0187130244206637 14,0.008642157986448138,0.15331223624944687,0.0,0.0,0.0,0.0,0.07454248818755149
-4.275218664169311,-9.567868179321 29,-6.0,0.0139117279462516 31,0.008052512646677447,0.15776021708548 07,0.0,0.0,0.0,0.0,0.08110717469081283,0.
-4.673508277893067,-9.950409634590148,-6.0,0.0100081701856106 52,0.007534644074969652,0.15230020935833455,0.0,0.0,0.0,0.0,0.08982257375121117,
-5.019800169944763,-10.325073850486755,-6.0,0.0069445127064827 83,0.007074361142154281,0.15554409326612 95,0.0,0.0,0.0,0.0,0.09807595687359572,
-5.319763863563338,-10.687696510314941,-6.0,0.0045483918574900 6,0.006673011286351608,0.15873746709525585,0.0,0.0,0.0,0.0,0.10348436322063208,
```

Fig. 31: RLHF algorithm training log example.



```
,critic_norm_entropy_max,critic_norm_entropy_min,duration,episode,episode_reward,labeled_feedback,step,total_feedback,true_episode_rew
8500437904,2.3144342140392498,10.559897348329 47,0.6568996989810 55,5.31415510177612 3,2.0,88.07188511663117,0.0,2000,0.0,34.81322622055905
995803833,2.0986941688606 76,8.378149741172791,0.4337112506330013,5.174718856811523,3.0,168.3139664872433,0.0,3000,0.0,115.09759884866399
41177018880844,2.1101073583364487,7.6433456320762 63,0.4168136655688286,5.309820652080 57,4.0,217.55743036573404,0.0,4000,0.0,199.52620085
93580412865,2.1429701802730 56,7.285995594501495,0.4234282823291157,3.4684877395628 85,5.0,207.98341828759294,0.0,5000,0.0,102.74970533326
47129273415,2.1772894709110206,7.0490855601043 7,0.4327383720804596,5.6595940589904785,6.0,206.34785201733757,0.0,6000,0.0,109.21509245628
7.0,211.15089498710586,0.0,7000,0.0,0.0,147.26111151734577
0,77.31232360374997,128.0,8000,128.0,39.82888295696773
9.0,72.64007408518955,128.0,9000,128.0,31.287994769637127
0.0,56.57769234322768,128.0,10000,128.0,28.71020303147368
11.0,70.17585232120473,128.0,11000,128.0,29.34900054244335
12.0,83.47954731393838,128.0,12000,128.0,47.971290250577844
3.0,68.99782992792461,256.0,13000,256.0,35.132433807 51199
,14.0,98.46538407531625,256.0,14000,256.0,67.63184679016472
0,84.73205583891831,256.0,15000,256.0,31.182350295417034
0,103.67292780106072,256.0,16000,256.0,63.80600084815803
17.0,130.1991367999289 7,256.0,17000,256.0,91.62148992691125
,18.0,151.40816131639684,384.0,18000,384.0,74.08544832318582
,19.0,123.9705339485422,384.0,19000,384.0,65.41007076702931
20.0,141.61116632845 37,384.0,20000,384.0,90.75441603841053
21.0,184.27721086344172,384.0,21000,384.0,83.53678699337874
2.0,209.207496887655,384.0,22000,384.0,160.792650465967
8,23.0,162.98987465680693,512.0,23000,512.0,92.58177709959993
24.0,217.825588655076,512.0,24000,512.0,127.75498908559479
,25.0,194.7056710345205,512.0,25000,512.0,134.1652274870688
,26.0,248.89485469346982,512.0,26000,512.0,199.91681002966752
14,27.0,216.66005071565 45,512.0,27000,512.0,163.44181552964258
15,28.0,246.82027193493 56,640.0,28000,640.0,156.61257689652425
4,29.0,230.42051085876 3,640.0,29000,640.0,120.76272967055515
7,30.0,276.37660779412185,640.0,30000,640.0,197.17475429272724
1.0,248.95927527063 76,640.0,31000,640.0,190.83897738778484
32.0,298.83745917200577,640.0,32000,640.0,245.2988087923227
33.0,275.89159792537976,768.0,33000,768.0,254.84241865169108
5,34.0,244.14242855543853,768.0,34000,768.0,176.87121637920276
```

Fig. 32: RLHF algorithm training log example.

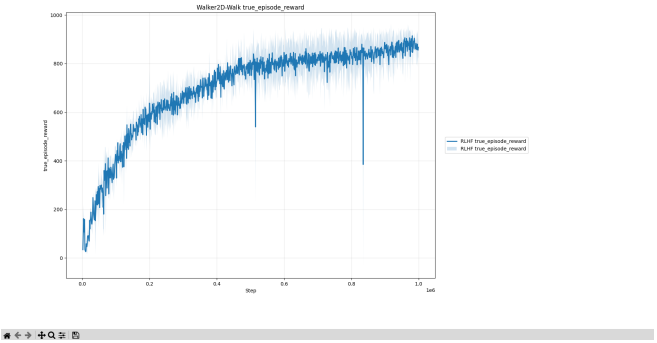


Fig. 39: The evaluation metric of true episode reward for SAC and RLHF algorithm on Walker2D-Walk environment.

C. 3. SAC and RLHF for Walker-Standup

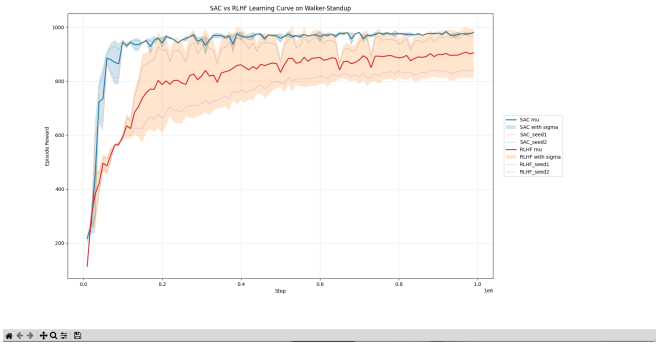


Fig. 40: The evaluation metric of testing procedure of episode reward for SAC and RLHF algorithm on Walker2D-Standup environment.

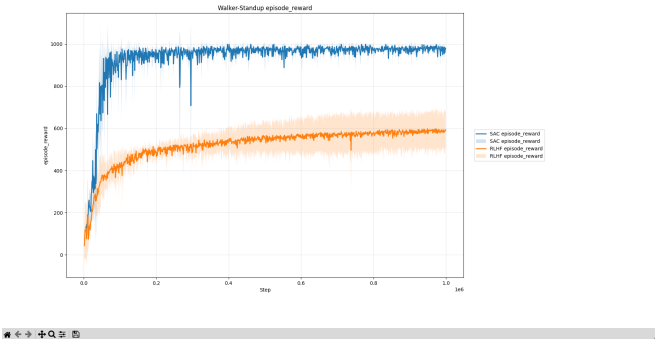


Fig. 41: The evaluation metric of training procedure of episode reward for SAC and RLHF algorithm on Walker2D-Standup environment.

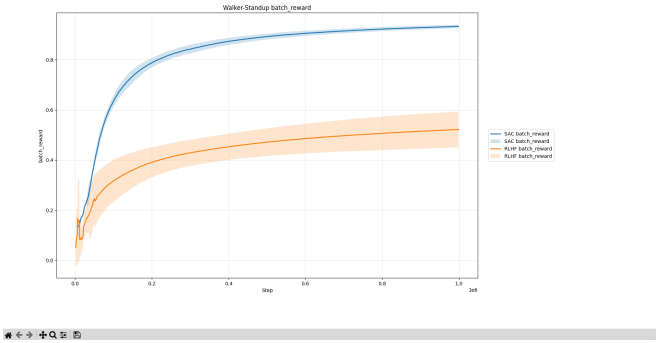


Fig. 42: The evaluation metric of batch reward for SAC and RLHF algorithm on Walker2D-Standup environment.

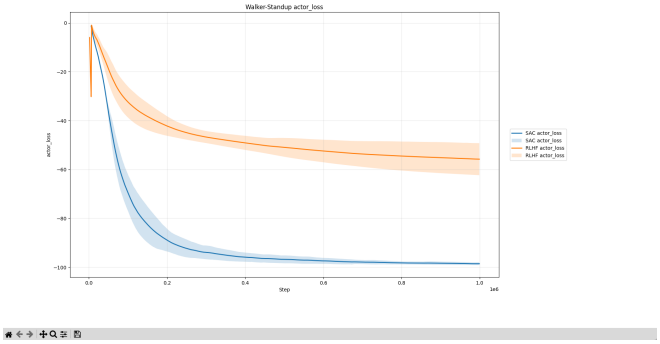


Fig. 43: The evaluation metric of actor loss for SAC and RLHF algorithm on Walker2D-Standup environment.

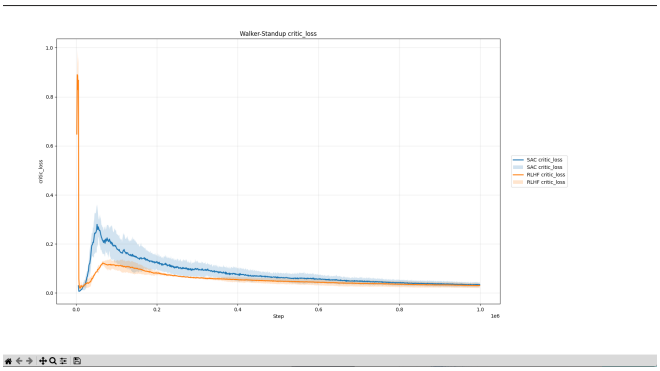


Fig. 44: The evaluation metric of critic loss for SAC and RLHF algorithm on Walker2D-Standup environment.

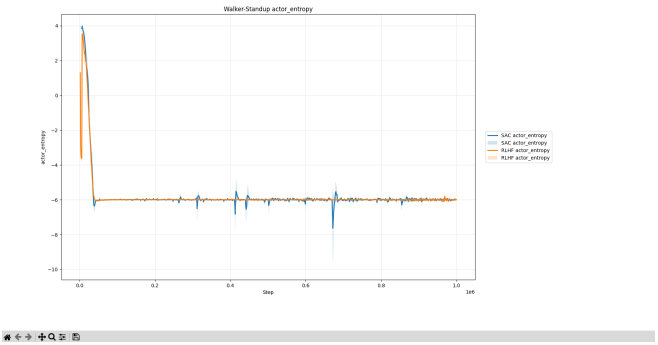


Fig. 45: The evaluation metric of actor entropy for SAC and RLHF algorithm on Walker2D-Standup environment.

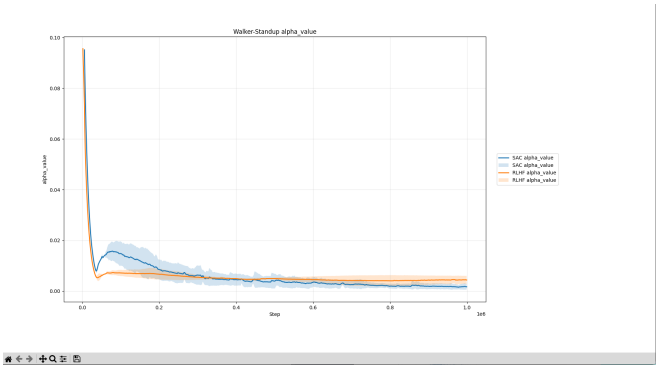


Fig. 46: The evaluation metric of alpha value for SAC and RLHF algorithm on Walker2D-Standup environment.

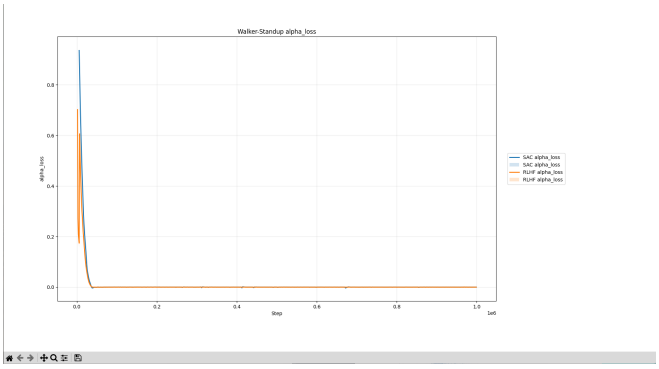


Fig. 47: The evaluation metric of alpha loss for SAC and RLHF algorithm on Walker2D-Standup environment.

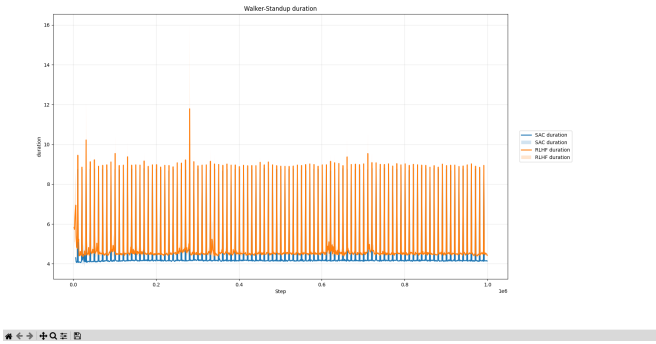


Fig. 48: The evaluation metric of duration for SAC and RLHF algorithm on Walker2D-Standup environment.

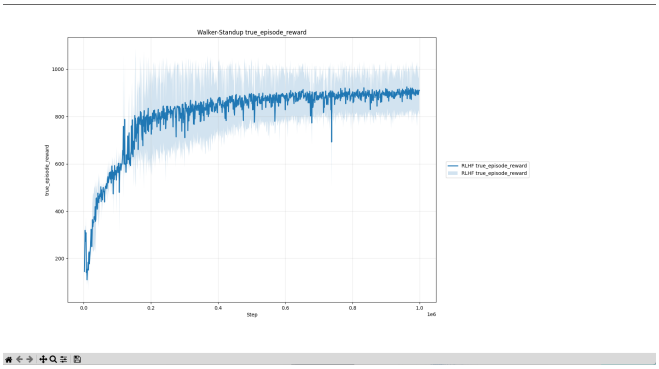


Fig. 49: The evaluation metric of true episode reward for SAC and RLHF algorithm on Walker2D-Standup environment.

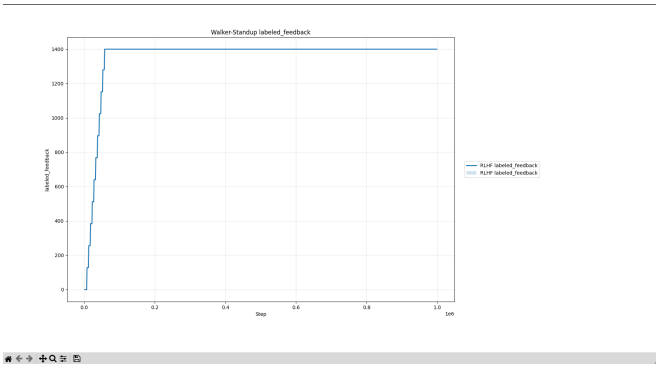


Fig. 50: The evaluation metric of labeled feedback for SAC and RLHF algorithm on Walker2D-Standup environment.

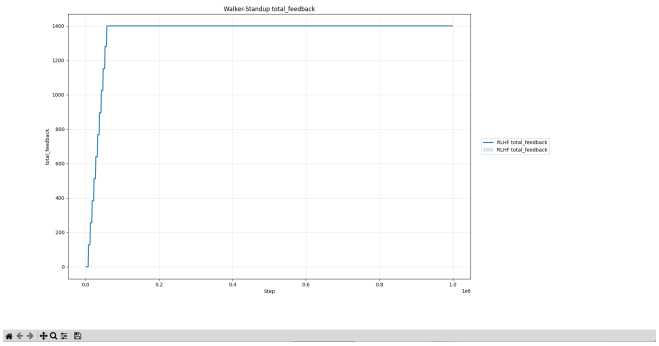


Fig. 51: The evaluation metric of total feedback for SAC and RLHF algorithm on Walker2D-Standup environment.

D. 4. SAC and RLHF for Humanoid-Standup

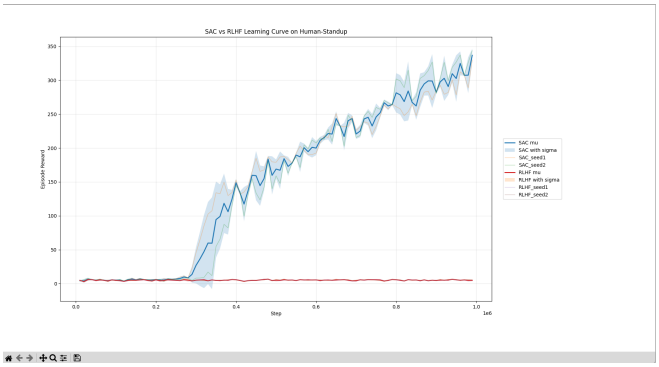


Fig. 52: The evaluation metric of testing procedure of episode reward for SAC and RLHF algorithm on Humanoid-Standup environment.

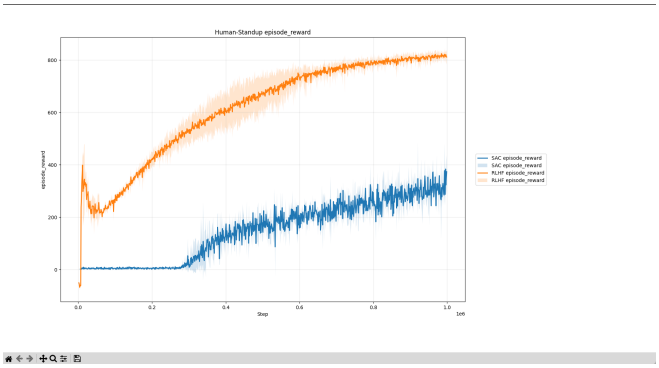


Fig. 53: The evaluation metric of training procedure of episode reward for SAC and RLHF algorithm on Humanoid-Standup environment.

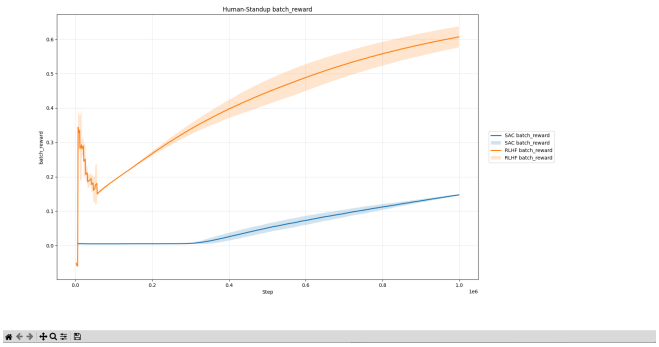


Fig. 54: The evaluation metric of batch reward for SAC and RLHF algorithm on Humanoid-Standup environment.

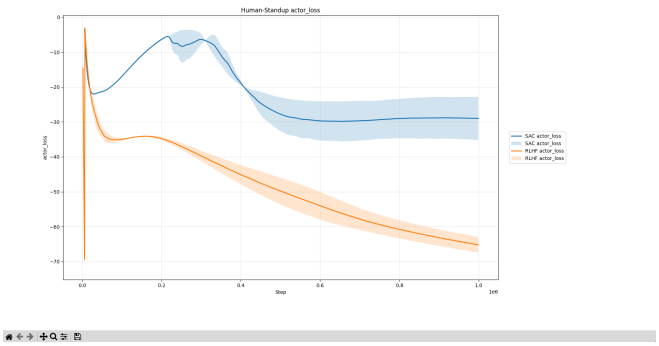


Fig. 55: The evaluation metric of actor loss for SAC and RLHF algorithm on Humanoid-Standup environment.

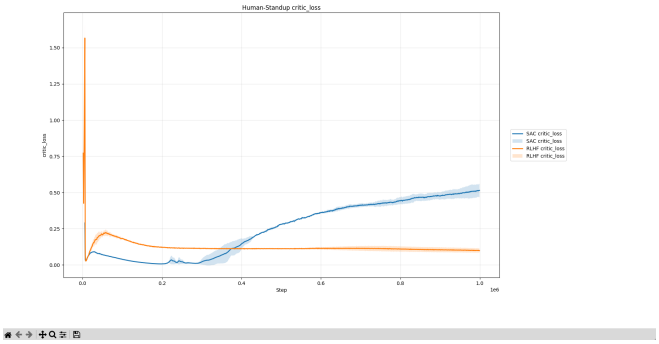


Fig. 56: The evaluation metric of critic loss for SAC and RLHF algorithm on Humanoid-Standup environment.

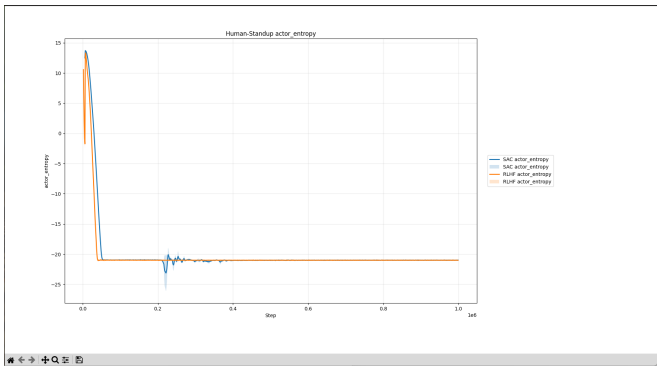


Fig. 57: The evaluation metric of actor entropy for SAC and RLHF algorithm on Humanoid-Standup environment.

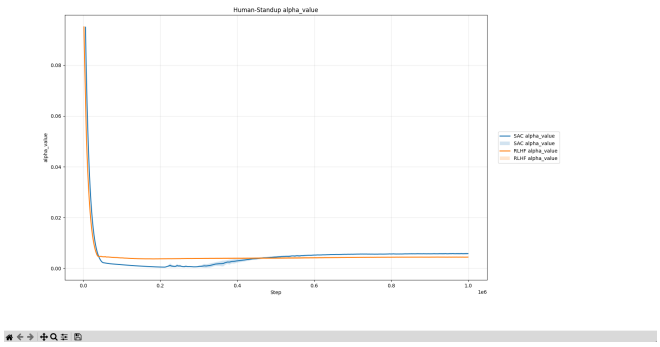


Fig. 58: The evaluation metric of alpha value for SAC and RLHF algorithm on Humanoid-Standup environment.

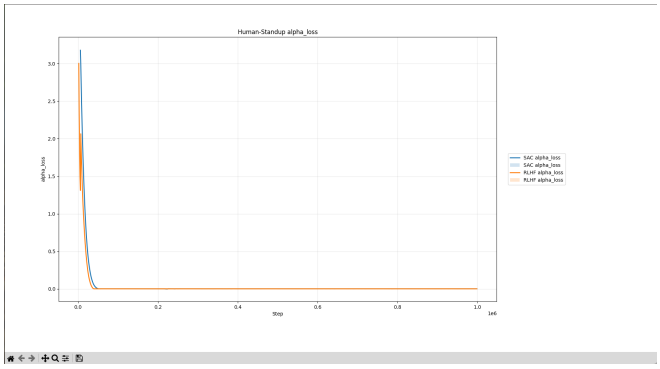


Fig. 59: The evaluation metric of alpha loss for SAC and RLHF algorithm on Humanoid-Standup environment.

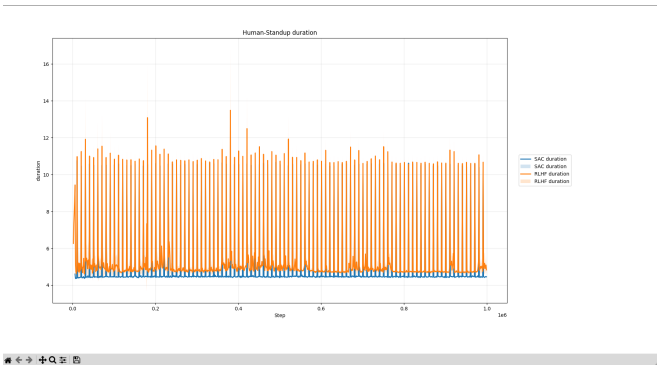


Fig. 60: The evaluation metric of duration for SAC and RLHF algorithm on Humanoid-Standup environment.

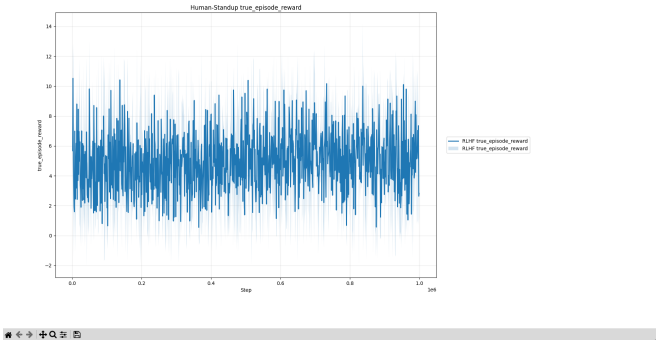


Fig. 61: The evaluation metric of true episode reward for SAC and RLHF algorithm on Humanoid-Standup environment.

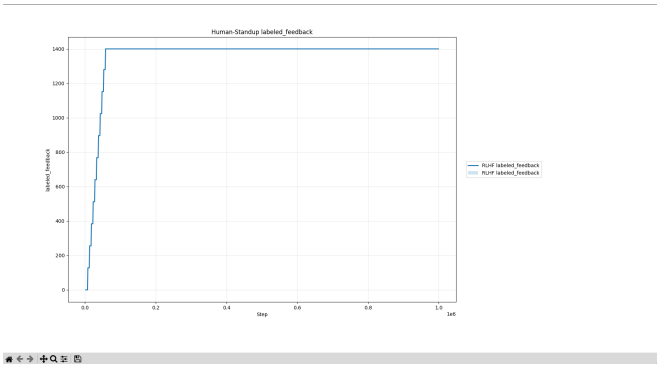


Fig. 62: The evaluation metric of labeled feedback for SAC and RLHF algorithm on Humanoid-Standup environment.

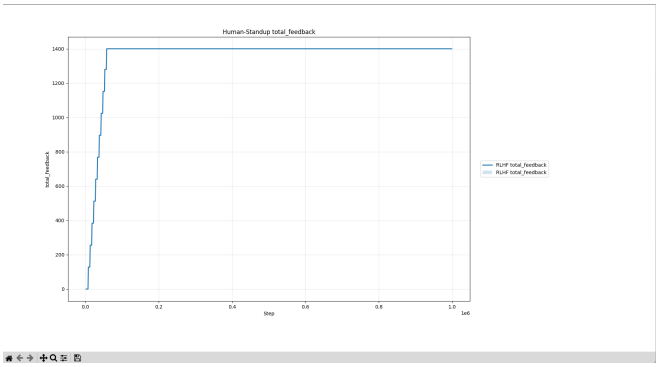


Fig. 63: The evaluation metric of total feedback for SAC and RLHF algorithm on Humanoid-Standup environment.

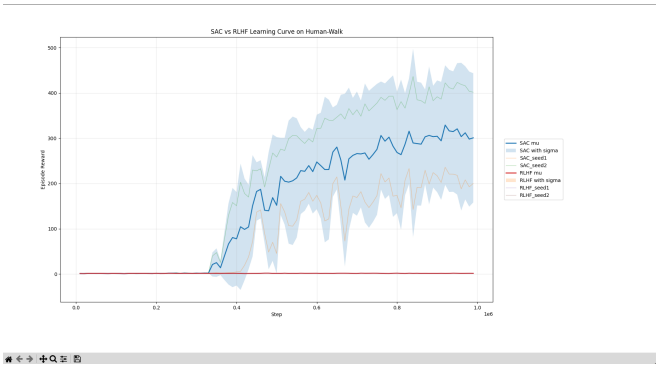


Fig. 64: The evaluation metric of testing procedure of episode reward for SAC and RLHF algorithm on Humanoid-Walk environment.

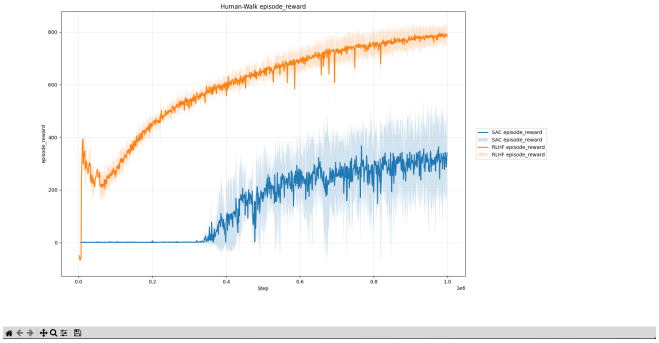


Fig. 65: The evaluation metric of training procedure of episode reward for SAC and RLHF algorithm on Humanoid-Walk environment.

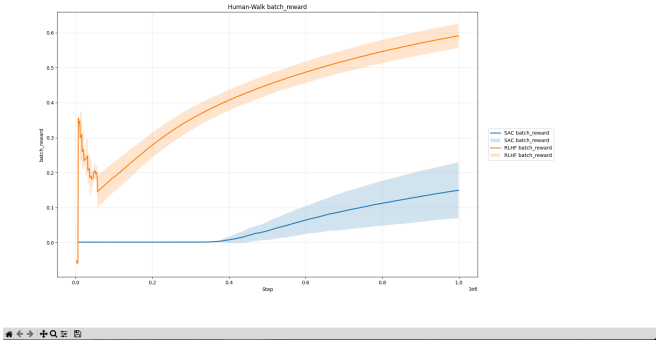


Fig. 66: The evaluation metric of batch reward for SAC and RLHF algorithm on Humanoid-Walk environment.

E. 5. SAC and RLHF for Humanoid-Walk

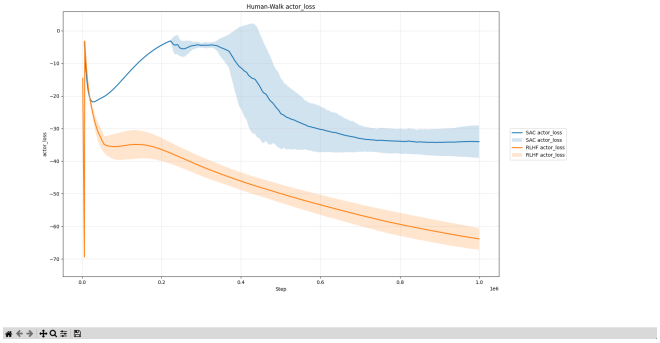


Fig. 67: The evaluation metric of actor loss for SAC and RLHF algorithm on Humanoid-Walk environment.

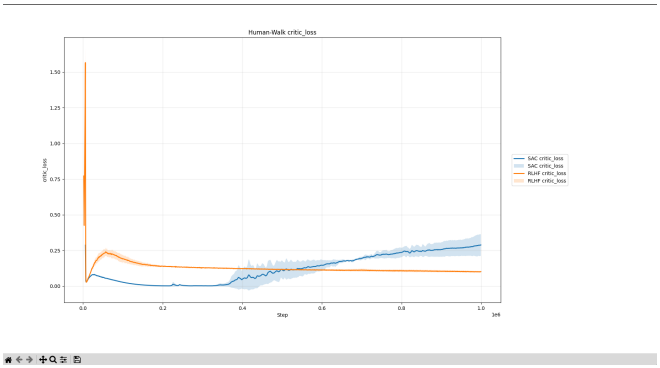


Fig. 68: The evaluation metric of critic loss for SAC and RLHF algorithm on Humanoid-Walk environment.

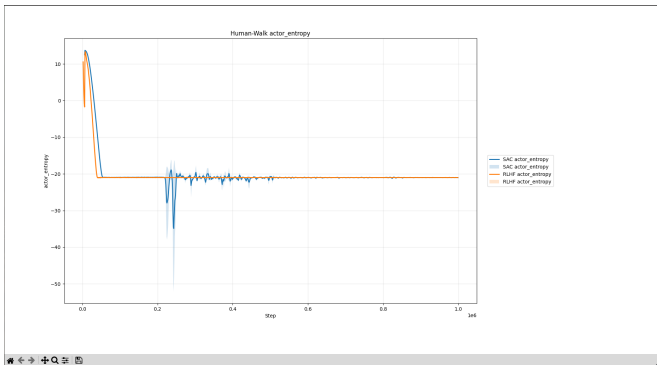


Fig. 69: The evaluation metric of actor entropy for SAC and RLHF algorithm on Humanoid-Walk environment.

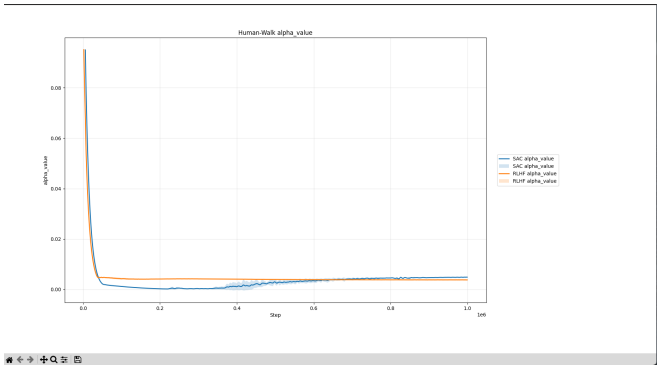


Fig. 70: The evaluation metric of alpha value for SAC and RLHF algorithm on Humanoid-Walk environment.

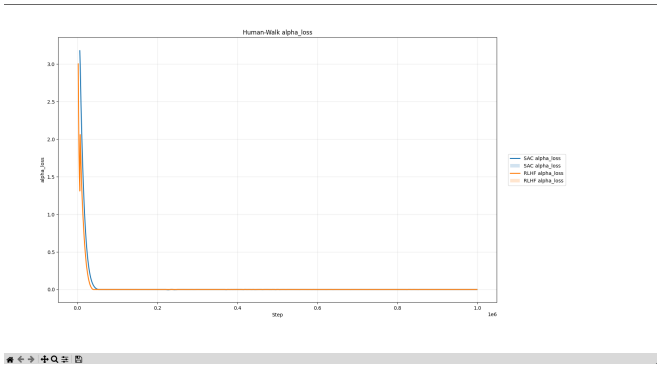


Fig. 71: The evaluation metric of alpha loss for SAC and RLHF algorithm on Humanoid-Walk environment.

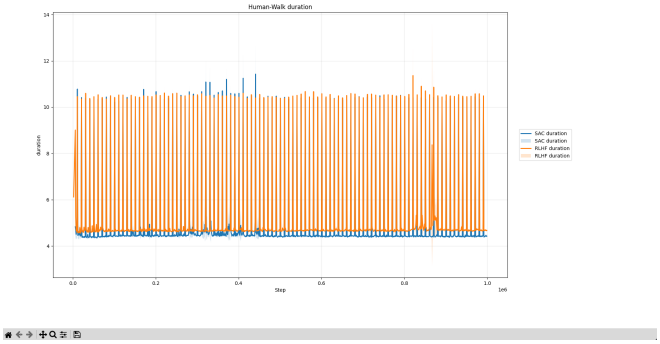


Fig. 72: The evaluation metric of duration for SAC and RLHF algorithm on Humanoid-Walk environment.

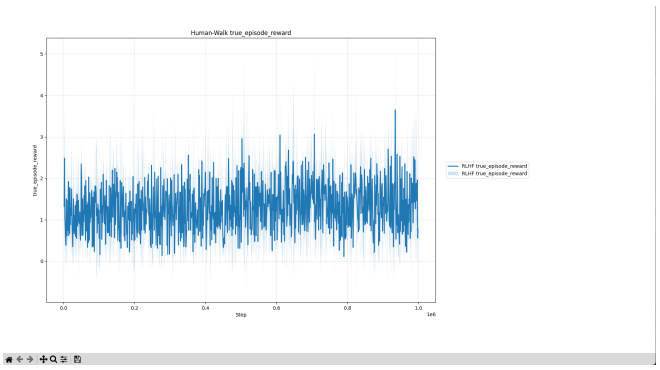


Fig. 73: The evaluation metric of true episode reward for SAC and RLHF algorithm on humanoid-Walk environment.

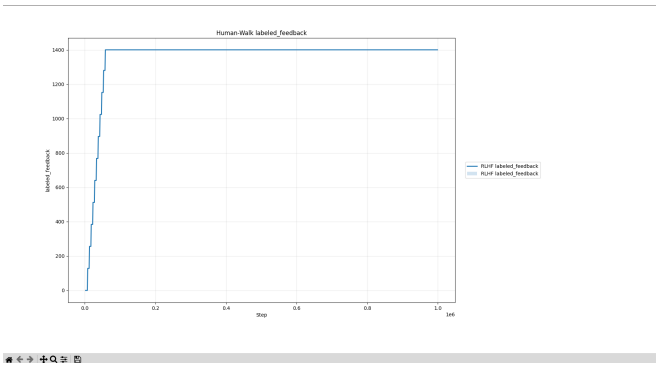


Fig. 74: The evaluation metric of labeled feedback for SAC and RLHF algorithm on Humanoid-Walk environment.

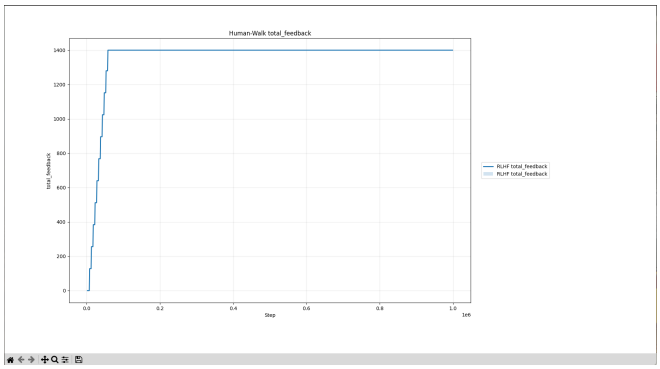


Fig. 75: The evaluation metric of total feedback for SAC and RLHF algorithm on Humanoid-Walk environment.

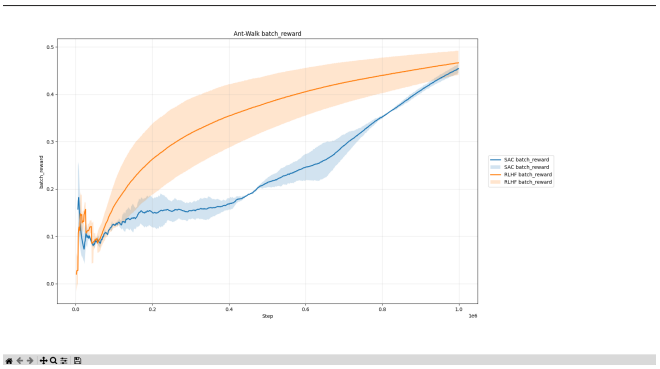


Fig. 78: The evaluation metric of batch reward for SAC and RLHF algorithm on Ant-Walk environment.

F. 6. SAC and RLHF for Ant-Walk

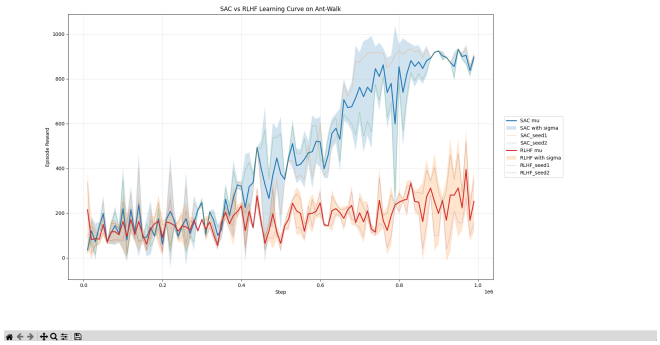


Fig. 76: The evaluation metric of testing procedure of episode reward for SAC and RLHF algorithm on Ant-Walk environment.

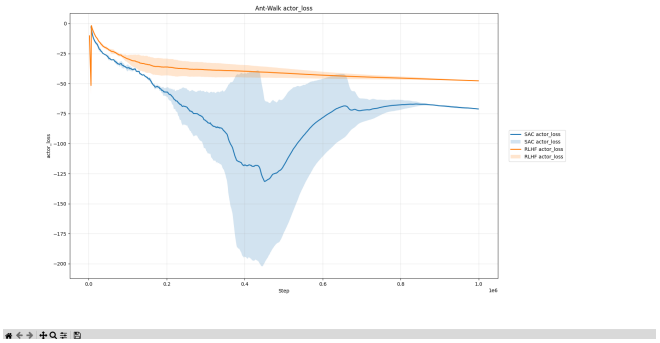


Fig. 79: The evaluation metric of actor loss for SAC and RLHF algorithm on Ant-Walk environment.

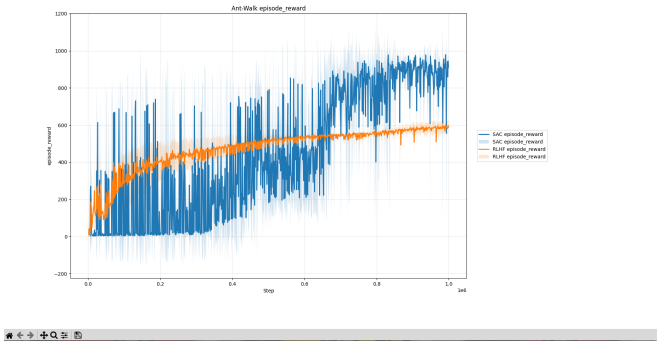


Fig. 77: The evaluation metric of training procedure of episode reward for SAC and RLHF algorithm on Ant-Walk environment.

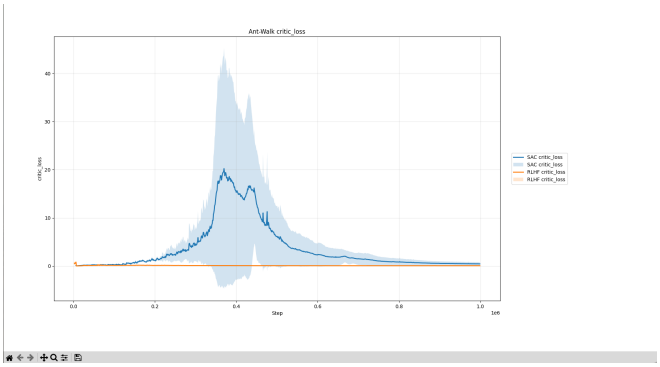


Fig. 80: The evaluation metric of critic loss for SAC and RLHF algorithm on Ant-Walk environment.

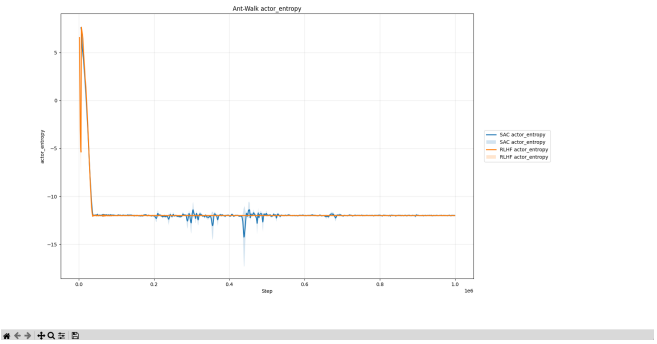


Fig. 81: The evaluation metric of actor entropy for SAC and RLHF algorithm on Ant-Walk environment.

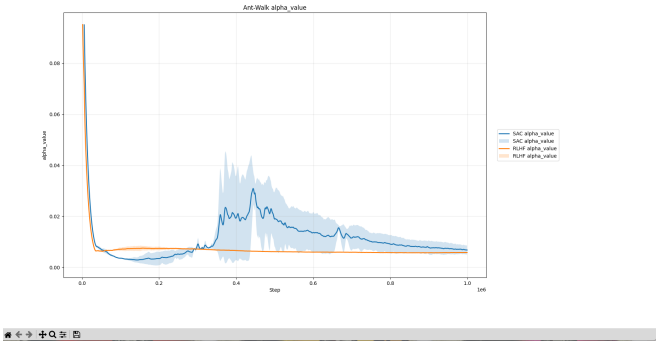


Fig. 82: The evaluation metric of alpha value for SAC and RLHF algorithm on Ant-Walk environment.

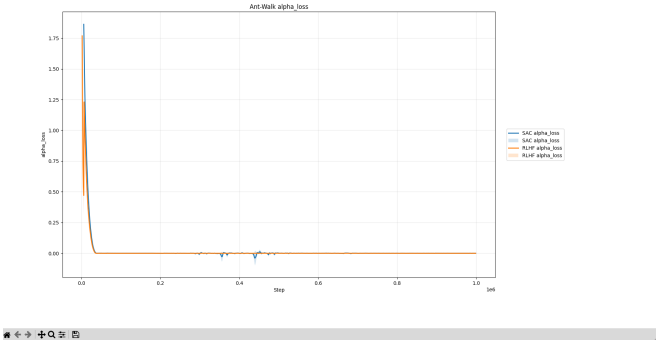


Fig. 83: The evaluation metric of alpha loss for SAC and RLHF algorithm on Ant-Walk environment.

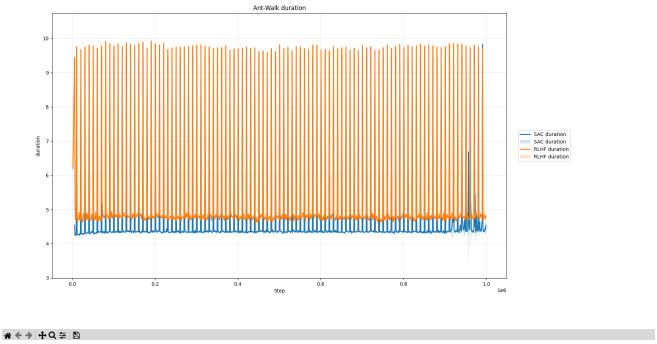


Fig. 84: The evaluation metric of duration for SAC and RLHF algorithm on Ant-Walk environment.

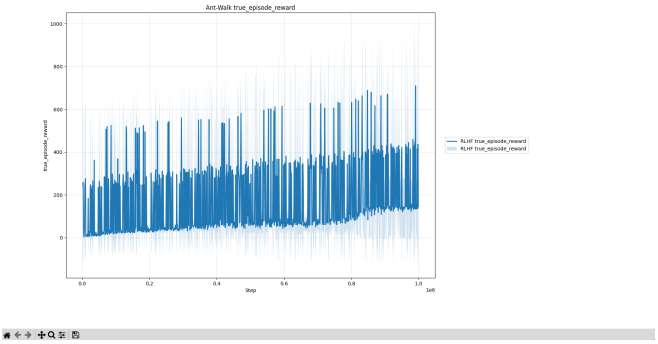


Fig. 85: The evaluation metric of true episode reward for SAC and RLHF algorithm on Ant-Walk environment.

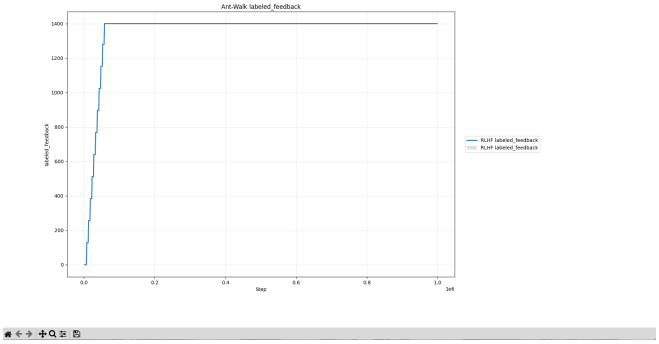


Fig. 86: The evaluation metric of labeled feedback for SAC and RLHF algorithm on Ant-Walk environment.

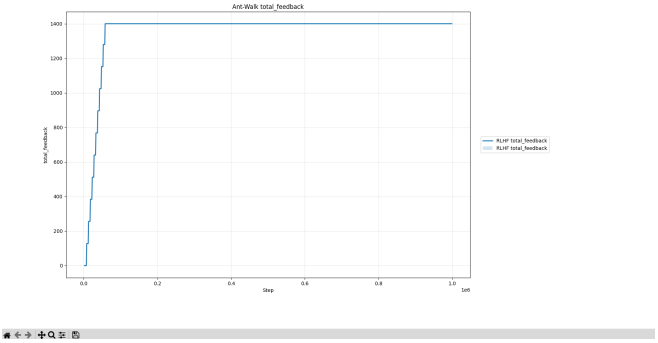


Fig. 87: The evaluation metric of total feedback for SAC and RLHF algorithm on Ant-Walk environment.

G. Algorithms Table

Algorithm 1 `SimTeacher`: Simulated human teachers

Require: Discount factor γ, rationality constant β, probability of making a mistake ϵ
Require: Skip threshold $\delta_{\texttt{skip}}$, equal threshold $\delta_{\texttt{equal}}$
Require: Pair of segments σ^0, σ^1
if $\max_{i\in\{0,1\}} \sum_t r\left(\mathbf{s}_t^i, \mathbf{a}_t^i\right) < \delta_{\texttt{skip}}$ **then** // SKIPPING QUERY
    $y \leftarrow \emptyset$
else if $\left|\sum_t r\left(\mathbf{s}_t^1, \mathbf{a}_t^1\right) - \sum_t r\left(\mathbf{s}_t^0, \mathbf{a}_t^0\right)\right| < \delta_{\texttt{equal}}$ **then** // EQUALLY PREFERABLE
    $y \leftarrow (0.5, 0.5)$
else if $\sigma^0 \succ \sigma^1 \sim P[\sigma^0 \succ \sigma^1; \beta, \gamma]$ **then** // SAMPLING PREFERENCES FROM (1)
    $y \leftarrow (1, 0)$ with probability of $1 - \epsilon$
    $y \leftarrow (0, 1)$ otherwise // MAKING A MISTAKE
else
    $y \leftarrow (0, 1)$ with probability of $1 - \epsilon$
    $y \leftarrow (1, 0)$ otherwise // MAKING A MISTAKE
end if
return y

Fig. 88: simulated supervisor algorithm.

Algorithm 3 Preference-based RL with reward learning

Require: frequency of teacher feedback K
Require: number of queries $N_{\texttt{query}}$ per feedback session
Initialize parameters of π_ϕ, $\widehat{r}_\psi$, a dataset of preferences $\mathcal{D} \leftarrow \emptyset$, and a buffer $\mathcal{B} \leftarrow \emptyset$
// EXPLORATION PHASE
$\mathcal{B}, \pi_\phi \leftarrow$ `EXPLORE()` in Algorithm 2
for each iteration **do**
    // REWARD LEARNING
    if iteration $\%\ K == 0$ **then**
        for m in $1 \ldots N_{\texttt{query}}$ **do**
            $(\sigma^0, \sigma^1) \sim$ `SAMPLE()` (see Section C) and query `SimTeacher` in Algorithm 1 for y
            Store preference $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\sigma^0, \sigma^1, y)\}$
        end for
        for each gradient step **do**
            Sample minibatch $\{(\sigma^0, \sigma^1, y)_j\}_{j=1}^D \sim \mathcal{D}$ and optimize $\mathcal{L}^{\texttt{Reward}}$ in (3) with respect to ψ
        end for
    end if
    // POLICY LEARNING
    for each timestep t **do**
        Collect $\mathbf{s}_{t+1}$ by taking $\mathbf{a}_t \sim \pi(\mathbf{a}_t|\mathbf{s}_t)$ and store transitions $\mathcal{B} \leftarrow \mathcal{B} \cup \{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}, \widehat{r}_\psi(\mathbf{s}_t))\}$
    end for
    for each gradient step **do**
        Sample random minibatch $\{(\tau_j)\}_{j=1}^B \sim \mathcal{B}$ and optimize RL objective function with respect to ϕ
    end for
    Reset $\mathcal{B} \leftarrow \emptyset$ if on-policy RL algorithm is used
end for

Fig. 89: RLHF algorithm